

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

QQ群号: 1031141115

第8章 输入/输出系统

- ◆ 8.1 I/O系统的性能
- ◆ 8.2 I/O系统的可靠性、可用性和可信性
- ◆ 8.3 廉价磁盘冗余阵列RAID
- ◆ 8.4 总线
- ◆ 8.5 通道处理机
- ◆ 8.6 I/O与操作系统

8.1 I/O系统的性能

- ◆ 输入/输出系统简称I/O系统
 - I/O设备
 - I/O接口（查询、中断、DMA）
 - I/O处理软件（I/O驱动程序）
- ◆ I/O系统是计算机系统中的一个重要组成部分
 - 完成计算机与外界的信息交换
 - 给计算机提供大容量的外部存储器
- ◆ 人们对I/O系统的作用和性能没有给予足够的重视
 - 人们更多地关注：CPU的性能。很多人甚至认为CPU的速度就是计算机的速度。
 - I/O设备通常被称为外围设备。外围的就似乎没那么重要了？

8.1 I/O系统的性能

- ◆ 系统的响应时间
 - 衡量计算机系统的一个更好的指标
 - 从用户输入命令开始，到得到结果所花费的时间。
 - 由两部分构成：I/O系统的响应时间，CPU的处理时间
- ◆ 问题：I/O系统性能与CPU性能间的关系？
- ◆ **误区：使用多进程技术可以忽略I/O性能对系统性能的影响**
- ◆ 事实是：
 - 多进程技术只能够提高系统吞吐率，并不能够减少系统响应时间。
 - 进程切换时可能需要增加I/O操作。
 - 可切换的进程数量有限，当I/O处理较慢时，仍然会导致CPU处于空闲状态。

I/O系统性能-举例

- ◆ 例8.1 假设一台计算机的I/O处理时间占响应时间的10%，当I/O性能保持不变，而对CPU的性能分别提高10倍和100倍时，该计算机系统的总体性能会发生什么样的变化？
- ◆ 解：假设改进前程序的执行时间为1个单位时间。
- ◆ **如果CPU的性能提高10倍**，程序的执行时间（包含I/O处理时间）减少为：

$$(1 - 10\%) / 10 + 10\% = 0.19$$

- ◆ 即整机性能只能提高到原来的约5倍，约50%的CPU性能被浪费在I/O处理上。
- ◆ **如果CPU的性能提高100倍**，程序的执行时间减少为：

$$(1 - 10\%) / 100 + 10\% = 0.109$$

- ◆ 这表示整机性能只能提高约10倍，约90%的性能被浪费在没有改进的I/O处理上。

8.1 I/O系统的性能

- ◆ 评价I/O系统性能的参数主要有：
 - 连接特性。哪些I/O设备可以和计算机系统相连接
 - I/O系统的容量。I/O系统可以容纳的I/O设备数
 - 响应时间
 - 吞吐率等
- ◆ 另一种衡量I/O系统性能的方法：
 - 考虑I/O操作对CPU的打扰情况。
 - 即考查某个进程在执行时，由于其他进程的I/O操作，使得该进程的执行时间增加了多少。

I/O系统的主要特点

◆ 异步性：

- 处理机和CPU之间以各自的速度执行，相互之间仅在必要时进行同步，而在两个同步点的时间段内，各自可以并行运行。

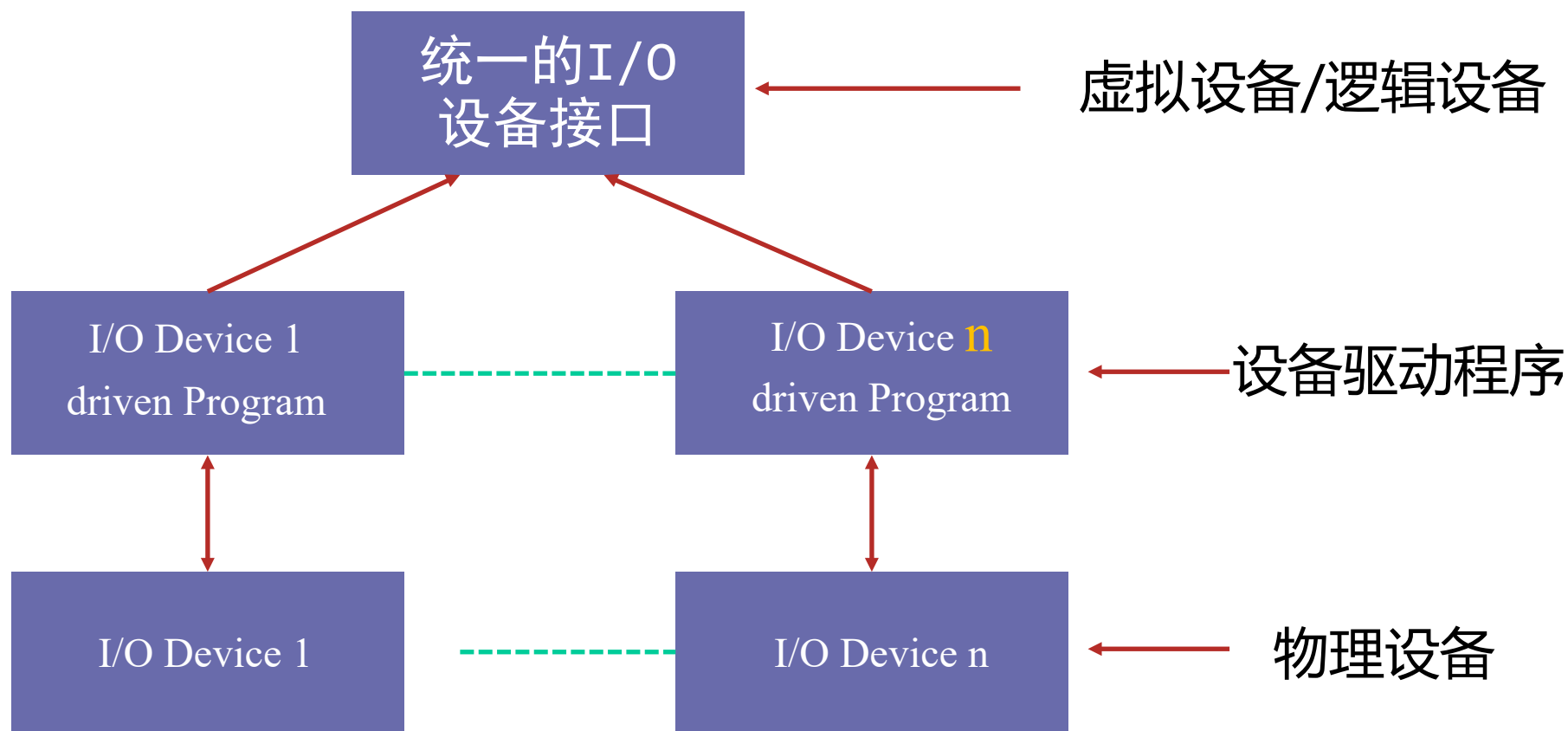
◆ 实时性：

- 在I/O设备提出中断、DMA等请求时，CPU要及时响应，完成必要的I/O操作或控制。例如：
Keyboard、Printer、COM、Mouse、定时器等。

◆ 与设备无关性：

- 通过制定统一的接口标准(物理接口、软件接口)，使得应用程序依据这一接口可以访问或支持各种I/O设备。

I/O系统的层次结构



8.2 I/O系统的可靠性、可用性和可信性

- ◆ 处理器性能已经很高，人们更加关注系统可靠性。
- ◆ 反映外设可靠性的参数有：
 - 可靠性 (Reliability)
 - 可用性 (Availability)
 - 可信性 (Dependability)
- ◆ **系统的可靠性**：系统从某个初始参考点开始一直连续提供服务的能力。
 - 用平均无故障时间MTTF (Mean Time To Failure) 来衡量。
 - MTTF的倒数就是系统的失效率。
 - **如果系统中每个模块的生存期服从指数分布，则系统整体的失效率是各部件的失效率之和。**

8.2 I/O系统的可靠性、可用性和可信性

- ◆ **系统的可用性**：系统正常工作的时间在连续两次正常服务间隔时间中所占的比率。

$$\text{可用性} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

- MTTR：平均修复时间（Mean Time To Repair）
- MTTF+MTTR：平均失效间隔时间MTBF（Mean Time Between Failure）
- ◆ **系统的可信性**：服务的质量。即在多大程度上可以合理地认为服务是可靠的。
 - 不可以度量

系统的可靠性-举例

- ◆ 例8.2 假设磁盘子系统的组成部件和它们的MTTF如下：
 - (1) 磁盘子系统由10个磁盘构成，每个磁盘的MTTF为1000000小时；
 - (2) 1个SCSI控制器，其MTTF为500000小时；
 - (3) 1个不间断电源，其MTTF为200000小时；
 - (4) 1个风扇，其MTTF为200000小时；
 - (5) 1根SCSI连线，其MTTF为1000000小时。
- ◆ 假定每个部件的**生存期服从指数分布**，同时假定各部件的故障是相互独立的，求整个系统的MTTF。

系统的可靠性-举例

◆ 解：

整个系统的失效率为：

$$\begin{aligned}\text{系统失效率} &= 10 \times \frac{1}{1000000} + \frac{1}{500000} + \frac{1}{200000} + \frac{1}{200000} + \frac{1}{1000000} \\ &= \frac{23}{1000000}\end{aligned}$$

系统的MTTF为系统失效率的倒数，即：

$$\text{MTTF} = \frac{1000000}{23} = 43500 \text{ 小时}$$

◆ 即将近5年。

8.3 廉价磁盘冗余阵列RAID

- ◆ 磁盘阵列DA (Disk Array)：使用多个磁盘（包括驱动器）的组合来代替一个大容量的磁盘。
 - 多个磁盘并行工作。
 - 以条带为单位把数据均匀地分布到多个磁盘上。
 - 交叉存放
 - 条带存放可以使多个数据读/写请求并行地被处理，从而提高总的I/O性能。
- ◆ 这里并行性有两方面的含义：
 - 多个独立的请求可以由多个盘来并行地处理。减少了I/O请求的排队等待时间
 - 如果一个请求访问多个块，就可以由多个盘合作来并行处理。提高了单个请求的数据传输率

8.3 廉价磁盘冗余阵列RAID

- ◆ 问题：阵列中磁盘数量的增加会导致磁盘阵列可靠性的下降。
 - 如果使用了N个磁盘构成磁盘阵列，那么整个阵列的可靠性将降低为单个磁盘的 $1/N$ 。
- ◆ 解决方法：在磁盘阵列中设置冗余信息盘
 - 当单个磁盘失效时，丢失的信息可以通过冗余盘中的信息重新构建。
- ◆ 廉价磁盘冗余阵列
 - Redundant Arrays of **Inexpensive** Disks
 - 磁盘冗余阵列（Redundant Arrays of **Independent** Disks）
 - 简称盘阵列技术
 - 1988年，Patterson教授首先提出

RAID提出

- ◆ 研究目的：效能与成本
 - 最初目的：当时CPU性能快速增长，CPU性能每年大约增长30~50%，而硬磁存储设备只能增长约7%。需找出一种新技术，在短期内立即提升存储效能来平衡计算机的运算能力。
 - 基于**廉价、并行架构思想，提高性能**
- ◆ 另外，研究小组也设计出容错（fault-tolerance）机制，逻辑数据备份（logical data redundancy），从而产生了RAID理论

RAID基本功能

(1) 高性能

- 对磁盘上的数据进行条带化，实现对数据按块存取，提高数据存取速度
- 对阵列中的几块磁盘同时读取，提高数据存取速度，同时提供大容量

(2) 大容量

- 多块独立磁盘组合成磁盘阵列，提供巨大存储容量

(3) 容错

- 通过镜像或者奇偶校验，实现对数据的冗余保护

8.3 廉价磁盘冗余阵列RAID

- ◆ 大多数磁盘阵列的组成可以由以下两个特征来区分
- ◆ 数据交叉存放的粒度
 - 细粒度磁盘阵列是在概念上把数据分割成相对较小的单位交叉存放。
 - 优点：所有I/O请求都能够获得很高的数据传输率。
 - 缺点：在任何时间，都只有一个逻辑上的I/O在处理当中，而且所有的磁盘都会因为为每个请求进行定位而浪费时间。
 - 粗粒度磁盘阵列是把数据以相对较大的单位交叉存放。
 - 多个较小规模的请求可以同时得到处理。
 - 对于较大规模的请求又能获得较高的传输率。
- ◆ 冗余数据的计算方法以及在磁盘阵列中的存放方式

8.3 廉价磁盘冗余阵列RAID

- ◆ 在磁盘阵列中设置冗余需要解决以下两个问题：
- ◆ **如何计算冗余信息？**
 - 大多都是采用**奇偶校验码**；
 - 也有采用**海明码**（Hamming code）或**RS纠删码**（Reed-Solomon码）
- ◆ **如何把冗余信息分布到磁盘阵列中的各个盘？**
- ◆ 有两种方法：
 - 把冗余信息**集中存放**在少数的几个盘中。
 - 把冗余信息**均匀地存放**到所有的盘中。
 - 能避免出现热点问题

RAID级别

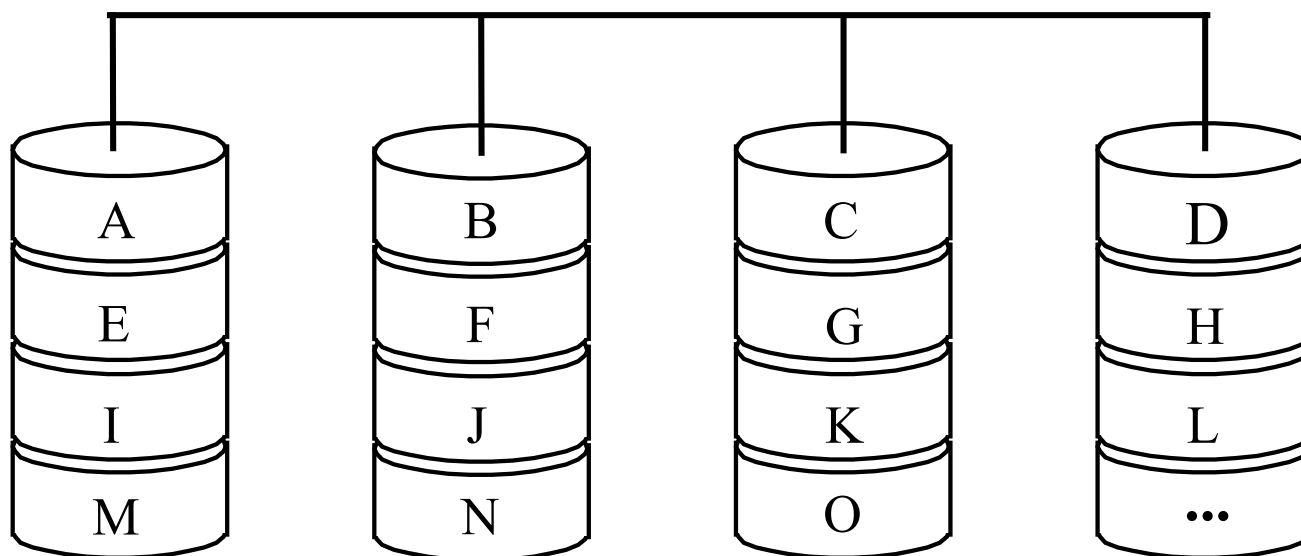
RAID级		数据 磁盘数	允许失效的 最大磁盘数	检测 磁盘数
0	非冗余	8	0	0
1	镜像	8	1	8
2	存储器式ECC	8	1	4
3	位交叉奇偶校验	8	1	1
4	块交叉奇偶校验	8	1	1
5	块交叉分布奇偶校验	8	1	1
6	P + Q冗余	8	2	2
7	Cache + 异步	8	2	2

RAID级别具有的共性

- ◆ RAID级别是具有如下共性的不同结构
 - 1、RAID由一组物理磁盘驱动器组成，操作系统视之为一个逻辑驱动器
 - 2、数据分布在一组物理磁盘上
 - 3、冗余信息被存储在冗余磁盘空间中，保证磁盘在万一损坏时可以恢复数据
 - 4、其中第2、3个特性在不同的RAID级别中的表现不同，RAID0不支持第3个特性。

RAID0

- ◆ 非冗余阵列，无冗余信息。
- ◆ 严格地说，它不属于RAID系列。
- ◆ 把数据切分成条带，以条带为单位交叉地分布存放到多个磁盘中。
- ◆ 2块以上硬盘即可，提高整个磁盘性能和吞吐量



RAID0 中的数据映射

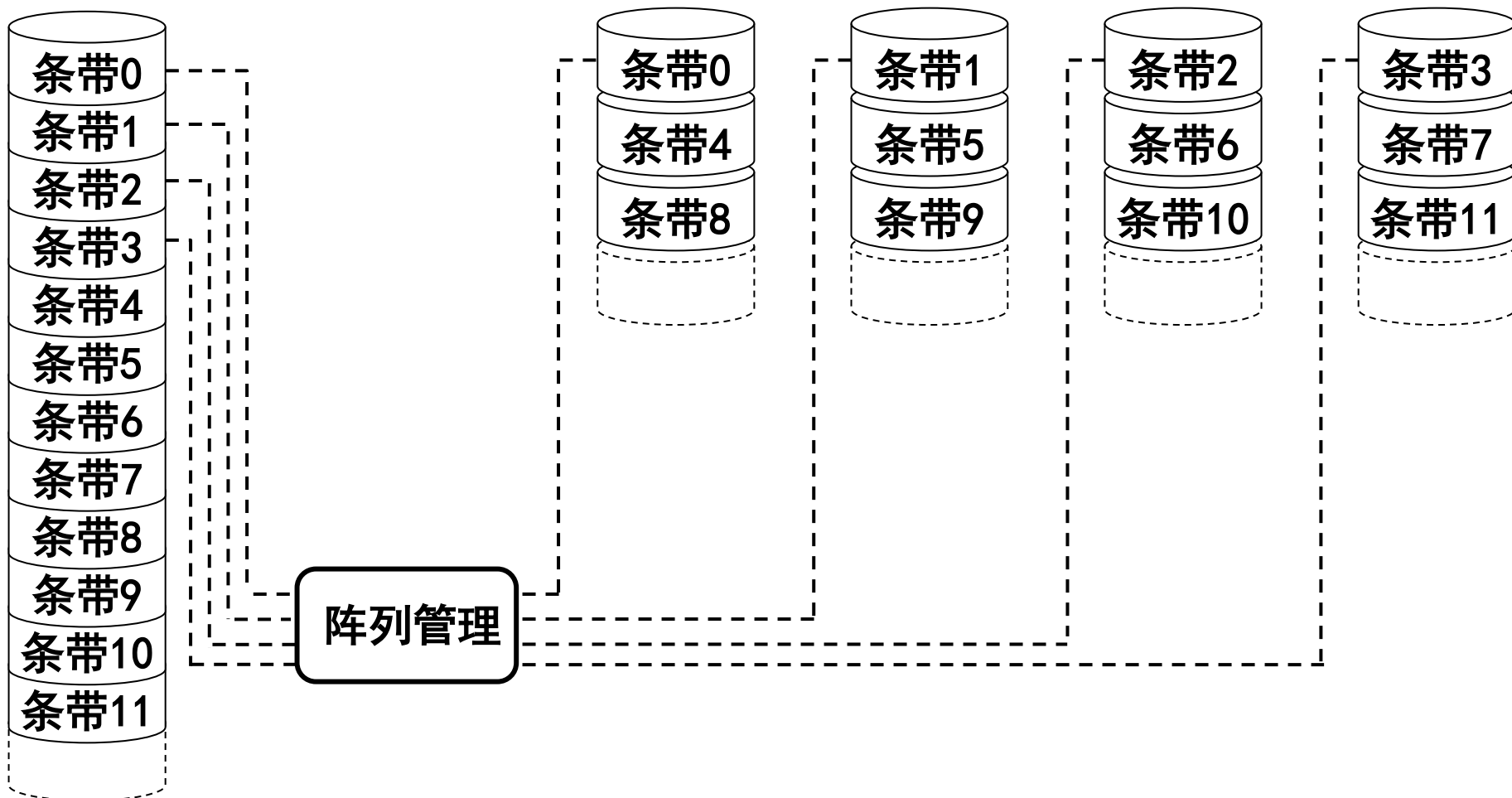
逻辑盘

物理盘 0

物理盘 1

物理盘 2

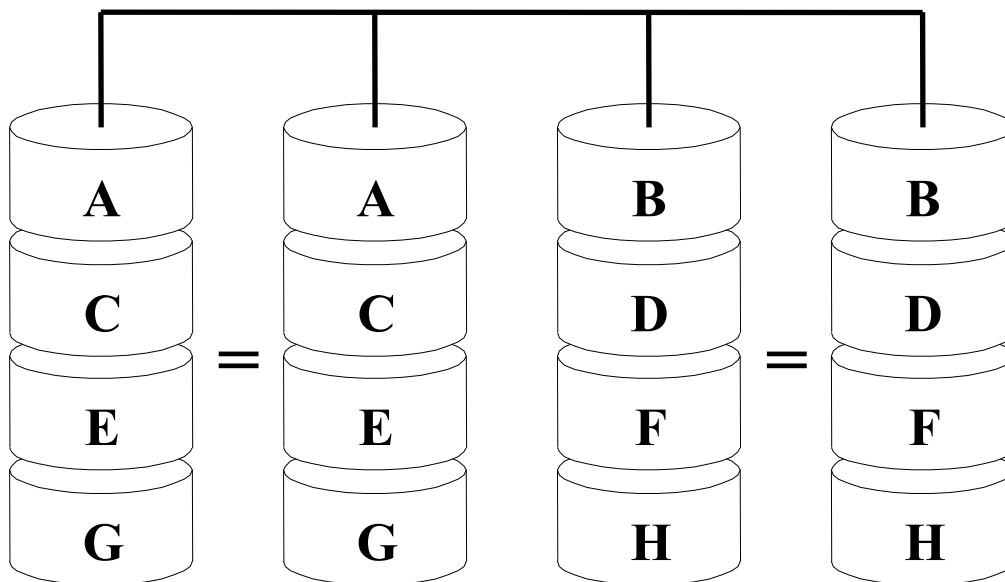
物理盘 3



RAID1

◆ 亦称镜像盘，使用双备份磁盘。

- 每当数据写入一个磁盘时，将该数据也写到另一个冗余盘，形成信息的两份复制品。
- 成本明显增加，磁盘利用率为50%
- 需要长时间同步镜像，影响系统性能
- 应用在保存关键性重要数据场合



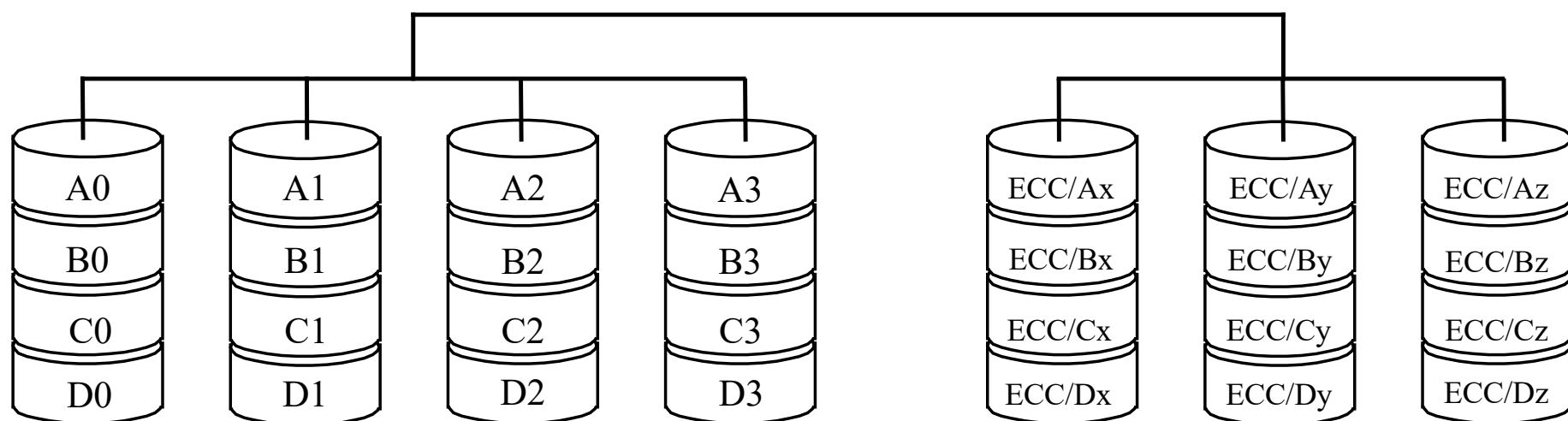
RAID1

◆ RAID 1特点：

- 读性能好：RAID1的性能能够达到RAID0性能的两倍；
- 写性能由写性能最差的磁盘决定，相对以后各级RAID来说，RAID1的写速度较快；
- 可靠性很高，数据的恢复很简单。
- 最昂贵的解决方法，物理磁盘空间是逻辑磁盘空间的两倍。

RAID2

- ◆ 位交叉式海明编码阵列
- ◆ 各个数据盘上的相应位计算海明校验码，编码位被存放在多个校验（ECC: Error Correct Code）磁盘的对应位上。
- ◆ 含4个数据盘的RAID2



字 A=A0~A3 字 B=B0~B3
字 C=C0~C3 字 D=D0~D3

字 A 的 ECC=A_x~A_z 字 B 的 ECC=B_x~B_z
字 C 的 ECC=C_x~C_z 字 D 的 ECC=D_x~D_z

RAID2

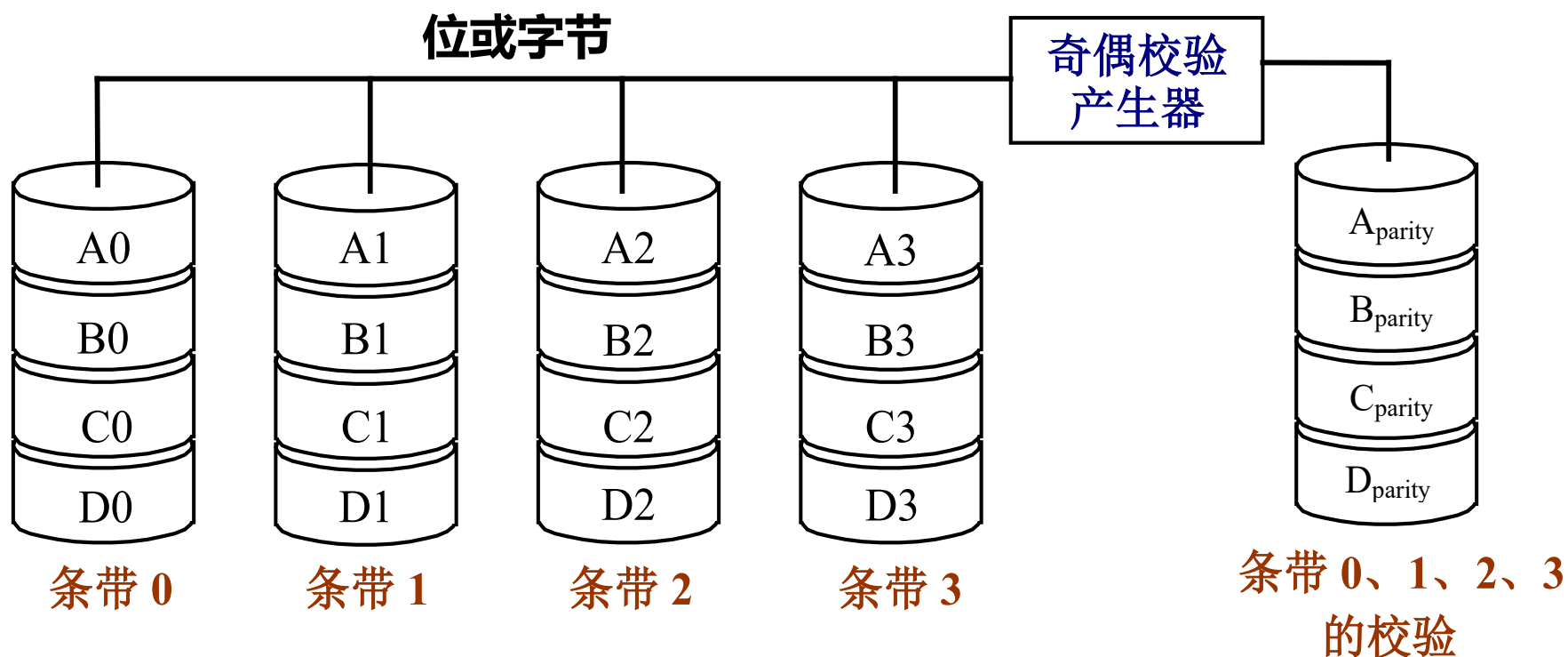
◆ RAID2的特点

- 每个数据盘存放所有数据字的一位
 - 位交叉存放
 - 并行存取，各个驱动器同步工作。
- 各个数据盘上的相应位计算海明校验码，编码位被存放在多个校验（ECC）磁盘的对应位上。
 - 使用海明编码来进行错误检测和纠正，数据传输率高。
- 需要多个磁盘来存放海明校验码信息，冗余磁盘数量与数据磁盘数量的对数成正比。
 - 冗余盘是用来存放海明码的，其个数为 $\log_2 m$ 级。
 - m ：数据盘的个数（也就是数据字的位数）
- 是一种在多磁盘易出错环境中的有效选择。
- 并未被广泛应用，目前还没有商业化产品。

RAID3

◆ 位交叉奇偶校验盘阵列

- 单盘容错并行传输：**数据以位或字节交叉存储**，奇偶校验信息存储在一个**专用硬盘**上。



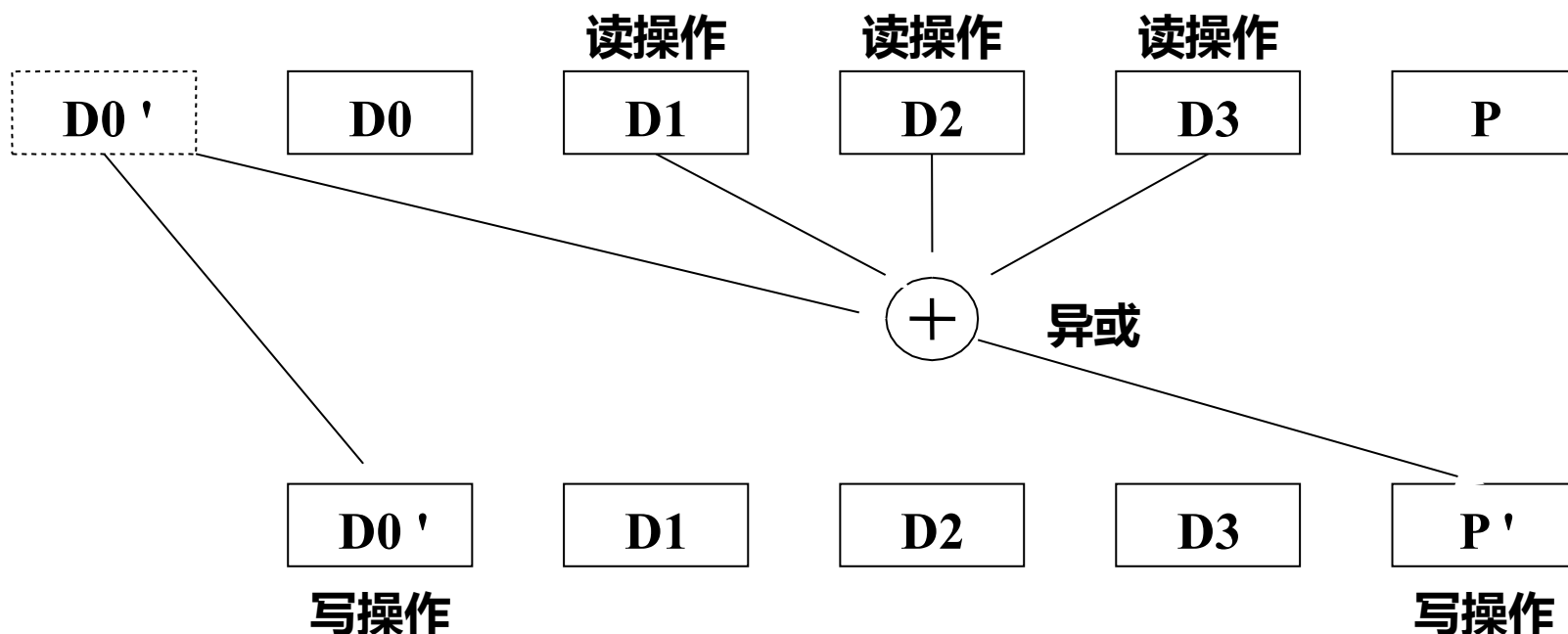
RAID3

◆ RAID3特点：

- 将磁盘分组，**读写要访问组中所有盘**，每组中有一个盘作为校验盘。
- 校验盘一般**采用奇偶校验**
- 简单理解：先将分布在各个数据盘上的一组数据加起来，将和存放在冗余盘上。一旦某一个盘出错，只要将冗余盘上的和减去所有正确盘上的数据，得到的差就是出错的盘上的数据。
- **缺点：恢复时间较长。**
- **适用于写操作少、读操作为主的场合**，比较适合大文件类型且安全性要求较高的应用，如WWW服务器、视频编辑、硬盘播出机、大型数据库等

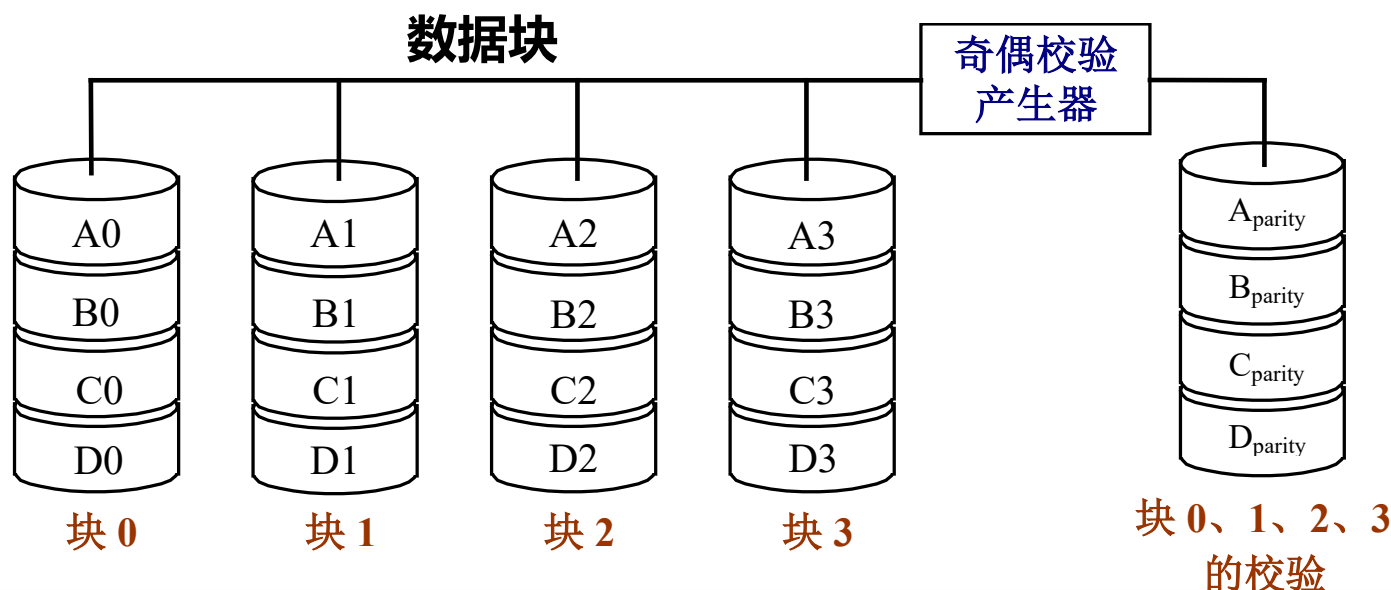
RAID3-举例

- ◆ 假定有4个数据盘和一个冗余盘。
- ◆ **读出数据**，一共需要5次磁盘读操作。
- ◆ **写数据**需要3次磁盘读和2次磁盘写操作。



RAID4

- ◆ 块交叉奇偶校验磁盘阵列
- ◆ **采用比较大的条带**，以块为单位进行交叉存放和计算奇偶校验。
 - 块大小可变，如文件
- ◆ 实现目标：能同时处理多个小规模访问请求



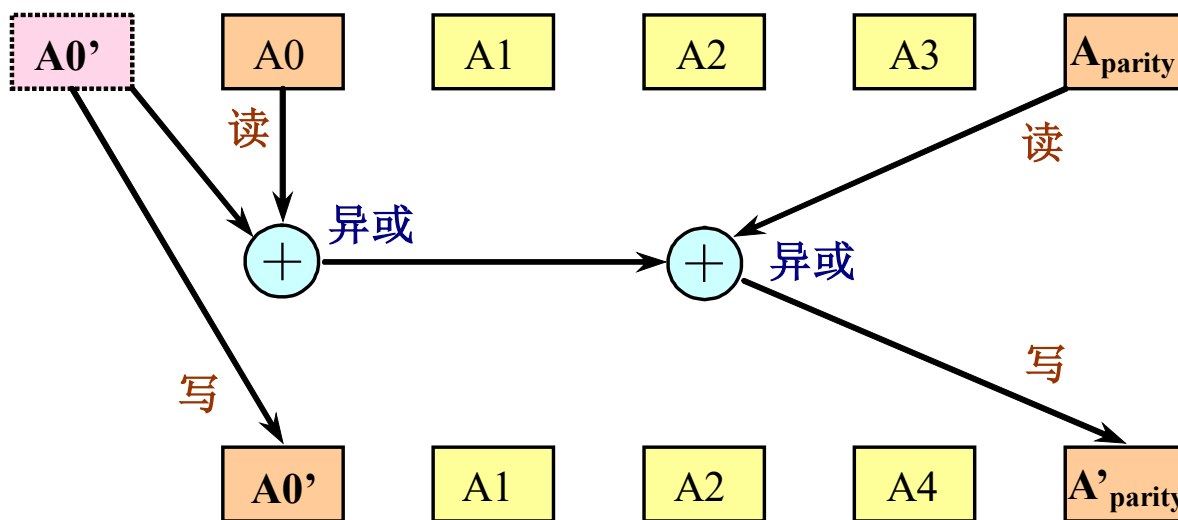
RAID4

- ◆ RAID4特点：
 - 冗余代价与RAID3相同。
 - 访问数据的方法与RAID3不同。
- ◆ 在RAID3中，一次磁盘访问将对磁盘阵列中的所有磁盘进行操作。
- ◆ **RAID4出现的原因**：希望使用较少的磁盘参与操作，以使磁盘阵列可以并行进行多个数据的磁盘操作
- ◆ 它对数据的访问是按数据块进行的，也就是按磁盘进行的，每次访问只是针对一个盘。
 - 每次只需访问数据所在的磁盘。
 - **仅在该磁盘出现故障时，才会去读校验盘，并进行数据的重建。**

RAID4

◆ RAID4读写特点：

- 假定有4个数据盘和一个冗余盘。
- 读出数据，对1个磁盘的1次读操作。
- 写数据需要2次磁盘读和2次磁盘写操作

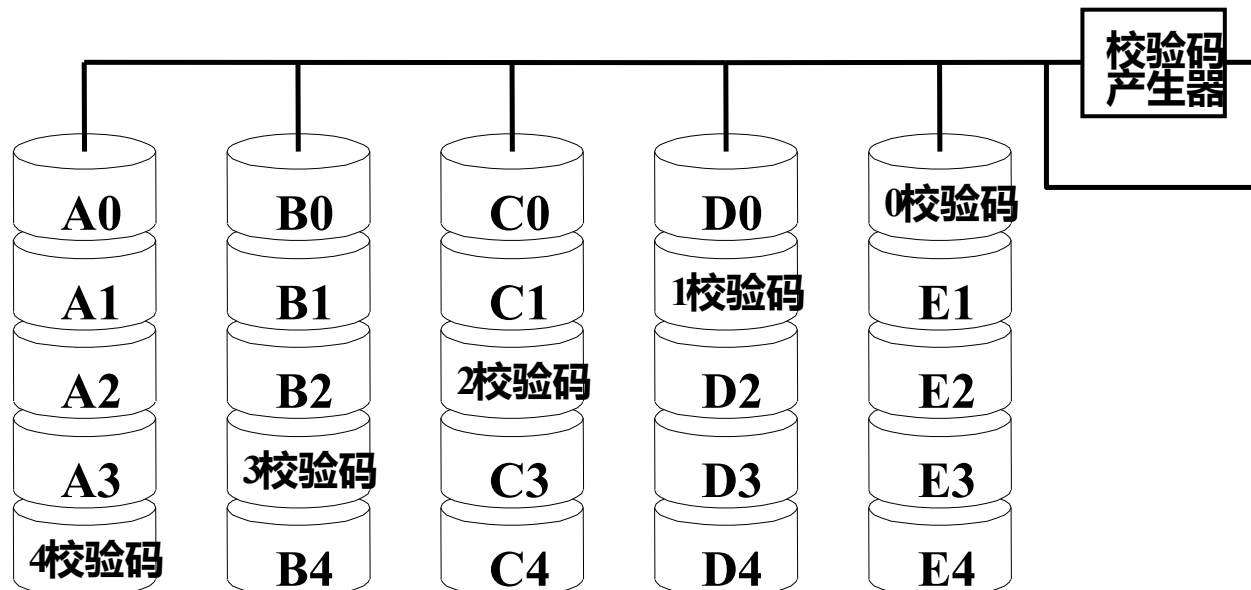


- RAID4能有效地处理小规模访问，快速处理大规模访问，校验空间开销比较小。

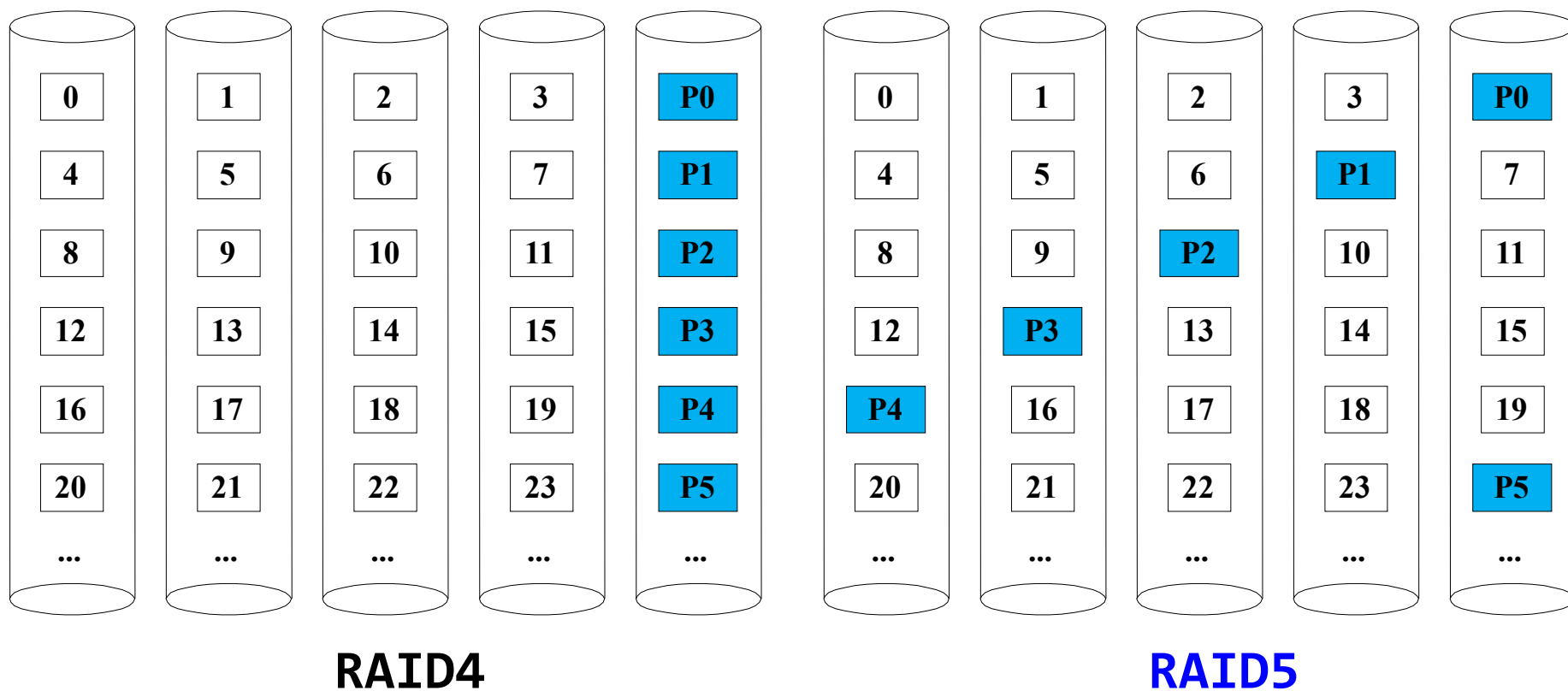
RAID5

◆ 块交叉分布奇偶校验磁盘阵列

- 数据以块交叉的方式存于各盘，无专用冗余盘，**奇偶校验信息均匀分布在所有磁盘**上。
- 不存在并发写操作时校验盘性能瓶颈问题
- 具备很好的扩展性，拥有更高容量及更高性能
- 是目前最常见RAID，数据中心大多采用它作为应用数据保护方案



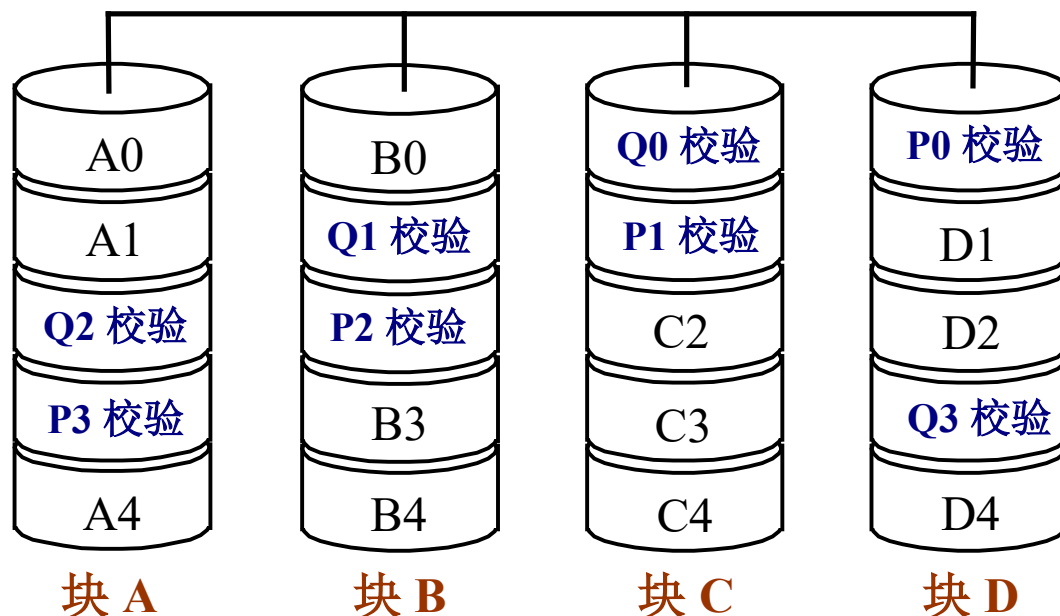
RAID4和RAID5中的信息分布



RAID 6

◆ 双维奇偶校验独立存取盘阵列

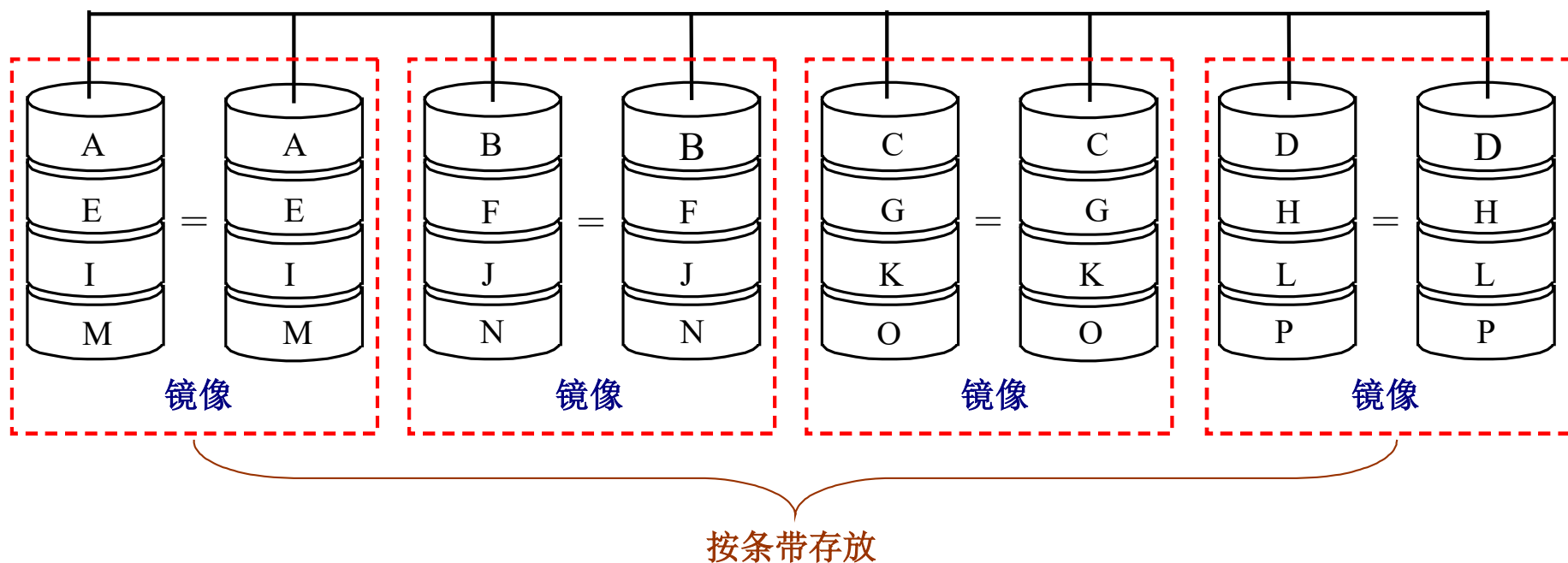
- P+Q双校验磁盘阵列
- 数据以块（块大小可变）交叉方式存于各盘，检、纠错信息均匀分布在所有磁盘上。
- 校验空间开销是RAID5的两倍
- 容忍两个磁盘出错
- 主要用于对数据安全等级要求非常高的场合
- 控制器比其他等级更复杂、更昂贵，写性能也较差



RAID10与RAID01

◆ RAID10又称为RAID1+0

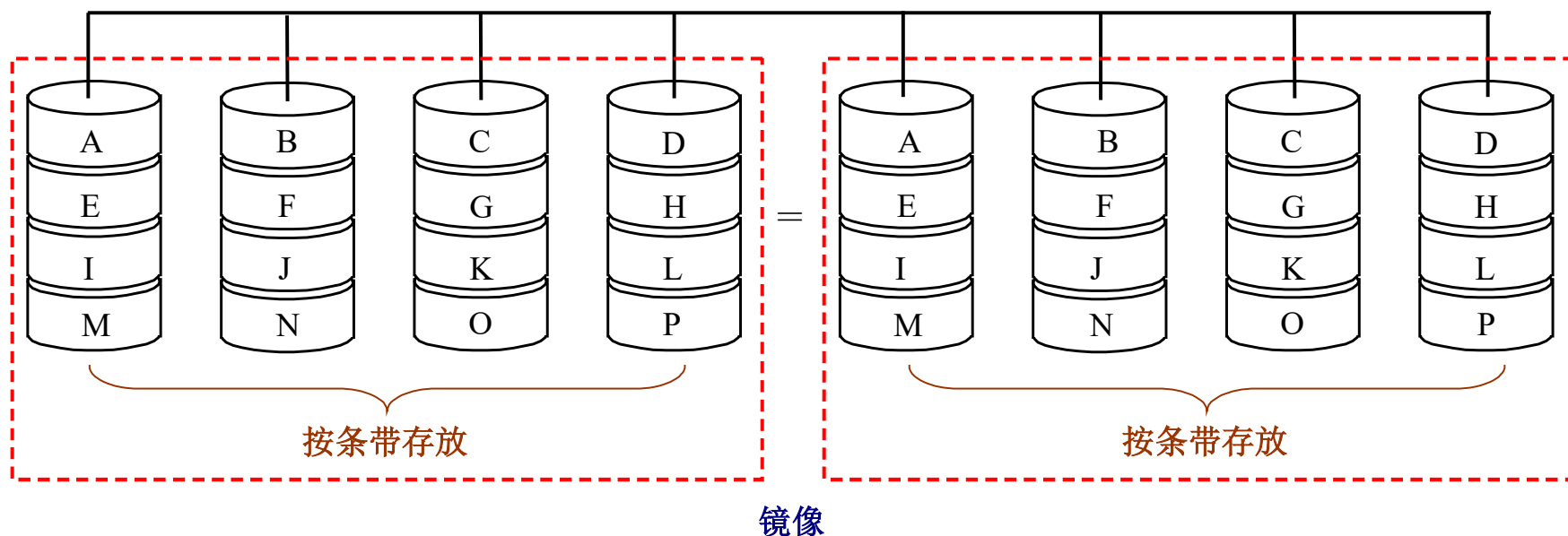
- 先进行镜像（RAID1），再进行条带存放（RAID0）



RAID10与RAID01

◆ RAID01又称为RAID0+1

- 先进行条带存放 (RAID0)，再进行镜像 (RAID1)



RAID的实现与发展

- ◆ 实现盘阵列的方式主要有三种：
 - **软件方式**：阵列管理软件由主机来实现。
 - 优点：成本低
 - 缺点：过多地占用主机时间，且带宽指标上不去。
 - **阵列卡方式**：把RAID管理软件固化在I/O控制卡上，从而可不占用主机时间，一般用于工作站和PC机。
 - **子系统方式**：一种基于通用接口总线的开放式平台，可用于各种主机平台和网络系统。

RAID的实现与发展

◆ 磁盘阵列技术研究的主要热点问题

- 新型阵列体系结构；
- RAID结构与其所记录文件特性的关系；
- 在RAID冗余设计中，综合平衡性能、可靠性和开销的问题
- 超大型磁盘阵列在物理上如何构造和连接。

◆ NVMe SSD?

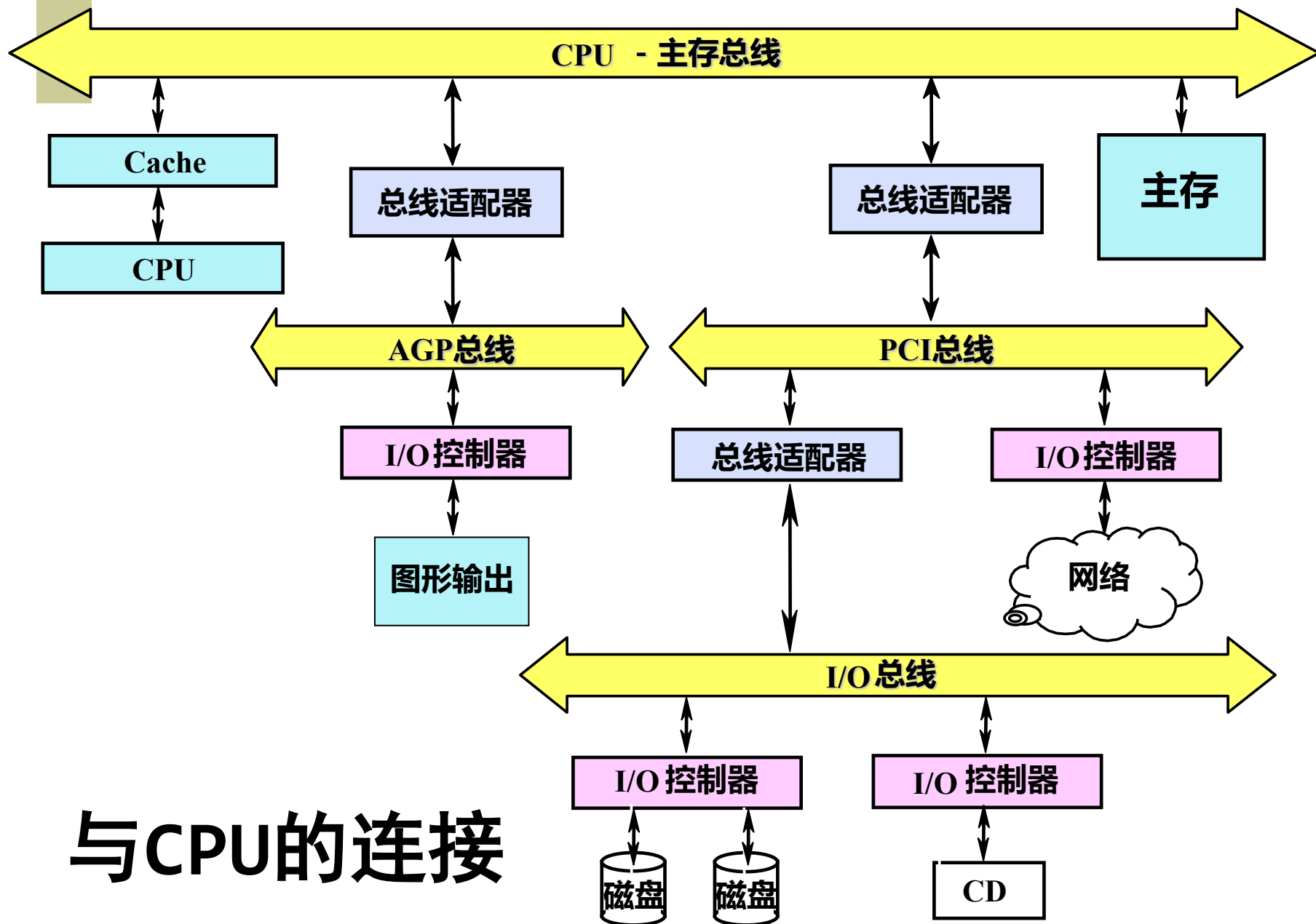
- RAID 2.0技术，通过增加一个虚拟化层（存储虚拟化）来将RAID与具体的硬盘解耦，从而改善RAID重建的数据时间。
- RAID-TP技术，其基于Erasure Code（简称EC）算法，将校验位做到支持1、2、3位可调，也就是说在同一个RAID组内，最大可支持3块硬盘同时故障，并保证数据不丢失，业务不中断。

8.4 总线

- ◆ 在计算机系统中，各子系统之间可以通过总线互相连接。
 - **优点**：成本低、多样性
 - **缺点**：它是由不同的外设分时共享的，形成了信息交换的瓶颈，从而限制了系统中总的I/O吞吐量。
- ◆ 8.4.1 总线的设计
- ◆ 总线设计存在很多技术难点
 - 总线上信息传送的速度极大地受限于各种物理因素
 - 我们一方面要求I/O操作响应快，另一方面又要求高吞吐量，这可能造成设计需求上的冲突。

设计总线时需要考虑的一些问题

特性	高性能	低价格
总线宽度	独立的地址和数据总线	数据和地址分时 共用同一套总线
数据总线宽度	越宽越快(例如: 64位)	越窄越便宜(例如: 8位)
传输块大小	块越大总线开销越小	单字传送更简单
总线主设备	多个(需要仲裁)	单个(无需仲裁)
分离事务	采用—分离的请求包和 回答包能提高总线带宽	不采用—持续连接成本 更低, 而且延迟更小
定时方式	同步	异步



与CPU的连接

8.4.2 与CPU的连接

◆ CPU对I/O设备的编址有两种方式

● 存储器映射I/O（也称为统一编址）

- 将一部分存储器地址空间分配给I/O设备，用load指令和store指令对这些地址进行读写将引起I/O设备的数据传输。
- 将一部分存储空间留出用于设备控制，对这一部分地址空间进行读写就是向设备发出控制命令。

● 给I/O设备独立编址

- 需要在CPU中设置专用的I/O指令来访问I/O设备。
- CPU需要发出一个标志信号来表示所访问的地址是I/O设备的地址。

◆ CPU与外部设备进行输入/输出的方式可分为4种

- 程序查询；中断；DMA；通道

8.5 通道处理机

- ◆ 程序控制、中断和DMA方式管理外围设备会引起两个问题：
 - **所有外设的输入/输出工作均由CPU承担**，CPU的计算工作经常被打断而去处理输入/输出的事务，不能充分发挥CPU的计算能力。
 - 低速设备：CPU执行一段子程序等
 - 高速设备：CPU对DMA初始化等
 - 大型计算机系统的外设虽然很多，但同时工作的机会不是很多。
 - 为每台设备都配置专用的DMA控制器，太浪费。
 - DMA接口处理速度很快，外围设备速度较慢或很慢
 - 多个设备共享1个DMA，这样DMA的管理将更加复杂
- ◆ 解决上述问题的方法：**采用通道处理机**

8.5.1 通道的作用和功能

◆ 目的：

- 将CPU进一步从I/O事务中脱离出来，使之具有更多的时间从事计算工作。提高关键部件的利用率
- 让DMA控制器能被多台设备共享，以提高附加硬件的利用率。

◆ 通道处理机（控制器）：

- 是一个具有输入输出处理控制的输入输出部件。
- 通道处理机有自己的指令，即通道命令。
- 能够执行有限输入输出指令，能够被多台IO设备共享（同时管理多台IO设备）的小型专用DMA处理机。

◆ 实例

- 采用通道的机型：IBM 360系列机，IBM370系列机，
- 采用微通道的机型：IBM公司研制的微型机和工作站。

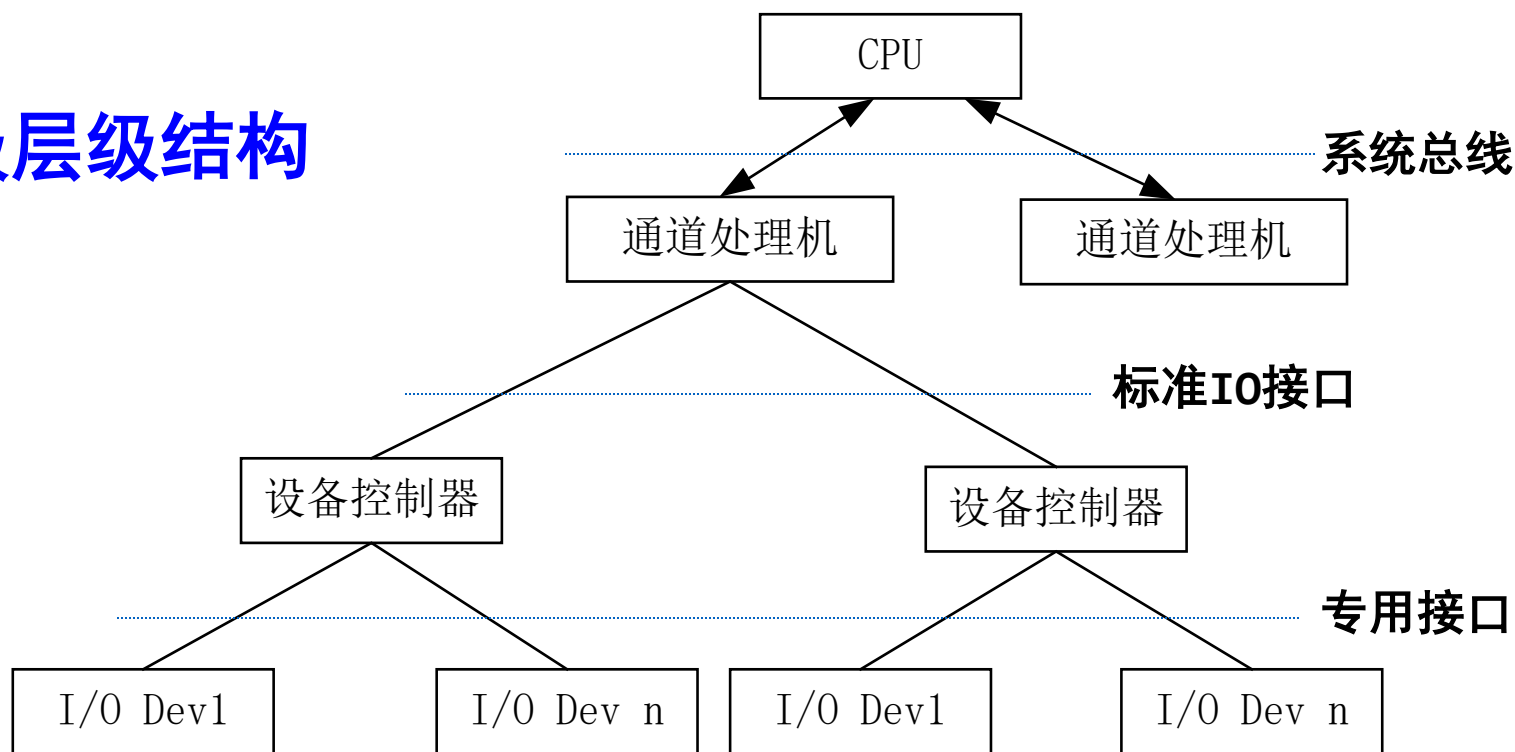
8.5.1 通道的作用和功能

- ◆ 一个典型的由**CPU、通道、设备控制器、外设**构成的**4级层次结构**的输入/输出系统。
- ◆ 负担原来CPU对外部设备的管理、控制和大部分输入输出工作。是一台同时能够被多台外围设备共享的DMA方式工作的小型专用处理机。
- ◆ 具体包括：
 - 管理所有按字节传输方式工作的低速和中速外围设备
 - 管理按数据块传输方式工作的高速外围设备
 - 对DMA方式工作的接口的初始化和后处理
 - 在OS指挥下，使得多处理机可以共享IO设备
 - 设备故障的检测和处理等。

通道处理机与CPU，设备控制器，I/O设备的关系

- ◆ 一台大型计算机系统中可以有多个通道，一个通道可以连接多个设备控制器，而一个设备控制器又可以管理一台或多台外围设备

- ◆ **四级层级结构**



通道的功能 1/2

- ◆ (1) 接收CPU发来的I/O指令，并根据指令要求选择指定的外设与通道相连接。
- ◆ (2) 执行通道程序
 - 从主存中逐条取出通道指令，
 - 对通道指令进行译码，
 - 根据需要向被选中的设备控制器发出各种操作命令。
- ◆ (3) 给出外设中要进行读/写操作的数据所在的地址
 - 磁盘存储器的柱面号、磁头号、扇区号等
- ◆ (4) 给出主存缓冲区的首地址
 - 该缓冲区存放从外设输入的数据或者将要输出到外设中去的数据。

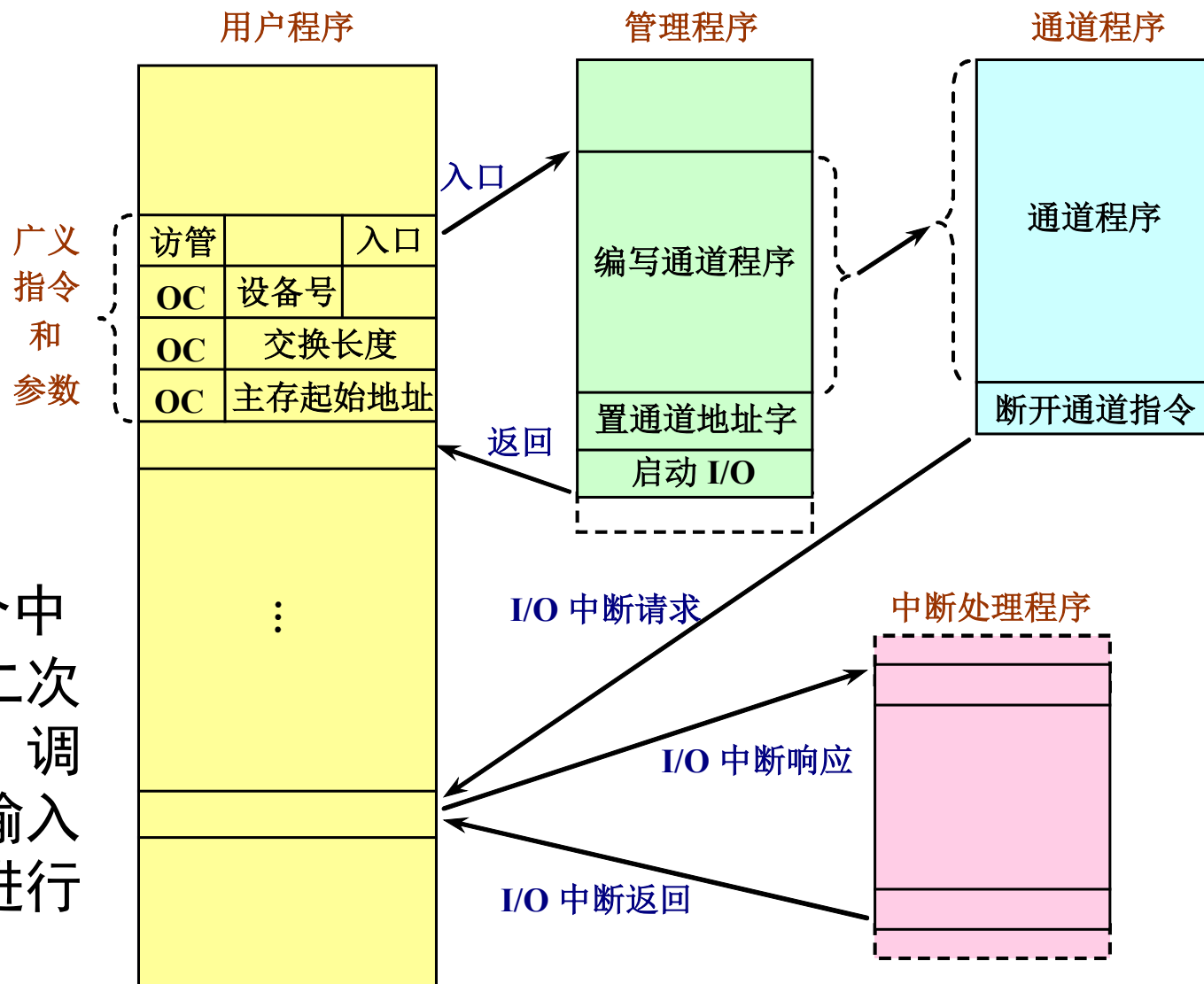
通道的功能 2/2

- ◆ (5) 控制I/O设备和MM间的数据交换。
 - 控制数据交换的个数
 - 对数据交换个数进行计数
 - 判断数据传送工作是否结束
- ◆ (6) 指定传送工作结束时要进行的操作
 - 将外设的中断请求及通道的中断请求送往CPU等。
- ◆ (7) 检查外设的工作状态是否正常，并将该状态信息送往主存指定单元保存。
- ◆ (8) 在数据传输过程中完成必要的格式变换
 - 把字拆分为字节，或者把字节装配成字等。

8.5.2 通道的工作过程

- ◆ 通道完成一次数据输入/输出的工作过程
- ◆ **1、在用户程序中使用访管指令进入管理程序，由管理程序生成一个通道程序，并启动通道。**
 - 用户在目标程序中设置一条广义指令，通过调用操作系统的管理程序来实现。
 - 管理程序根据广义指令提供的参数来编制通道程序。
 - 启动输入/输出设备指令是一条主要的输入/输出指令，属于特权指令。
- ◆ **2、通道处理机执行通道程序，完成指定的数据输入/输出工作。**
 - 通道处理机执行通道程序与CPU执行用户程序是并行的。
- ◆ **3、通道程序结束后向CPU发中断请求。**

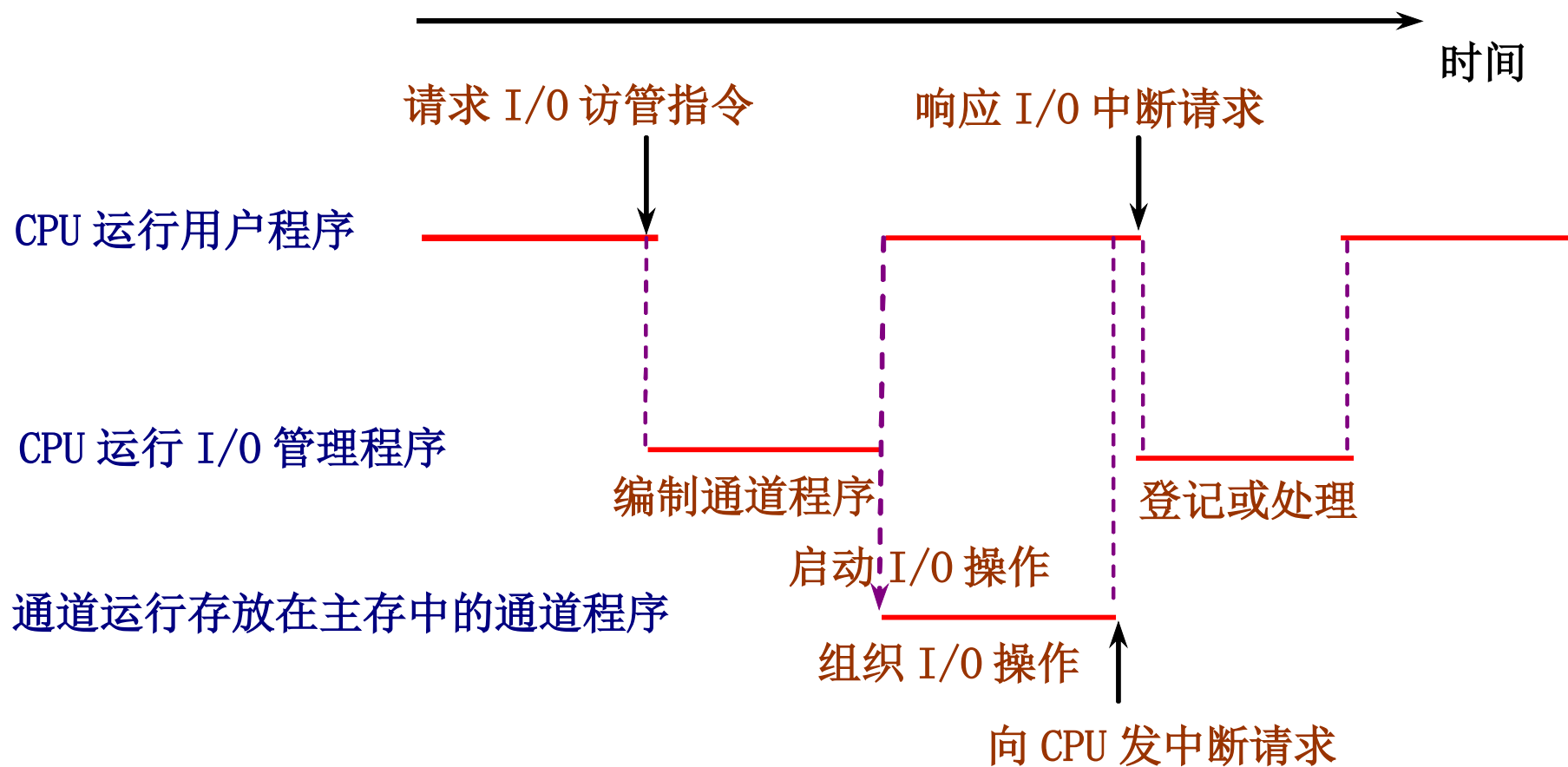
8.5.2 通道的工作过程



CPU响应这个中断请求后，第二次进入操作系统，调用管理程序对输入输出中断请求进行处理。

8.5.2 通道的工作过程

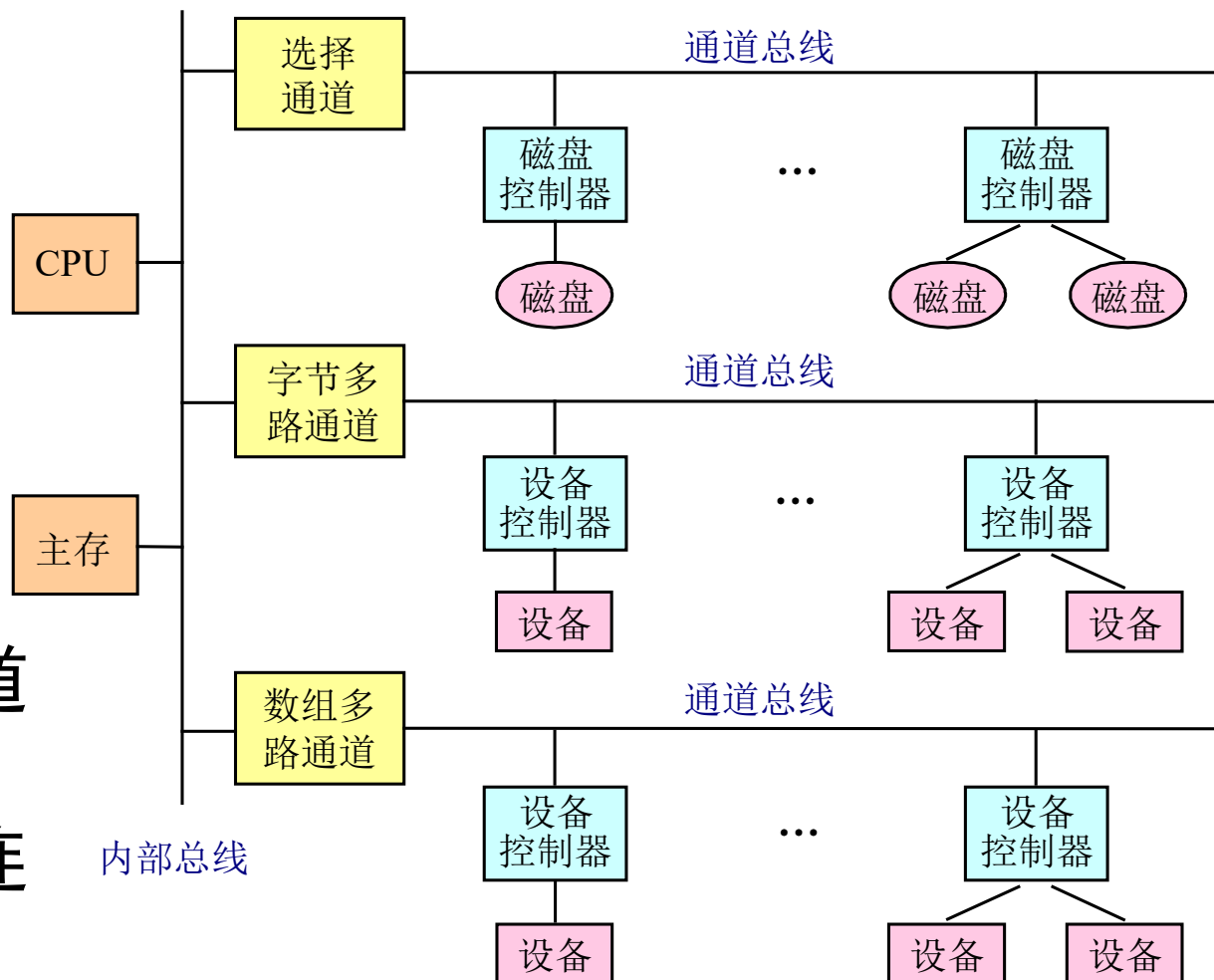
◆ CPU执行程序 and 通道执行程序的时间关系



8.5.3 通道的种类

◆ 根据信息传送方式的不同，将通道分为三种类型

- 字节多路通道
- 选择通道
- 数组多路通道

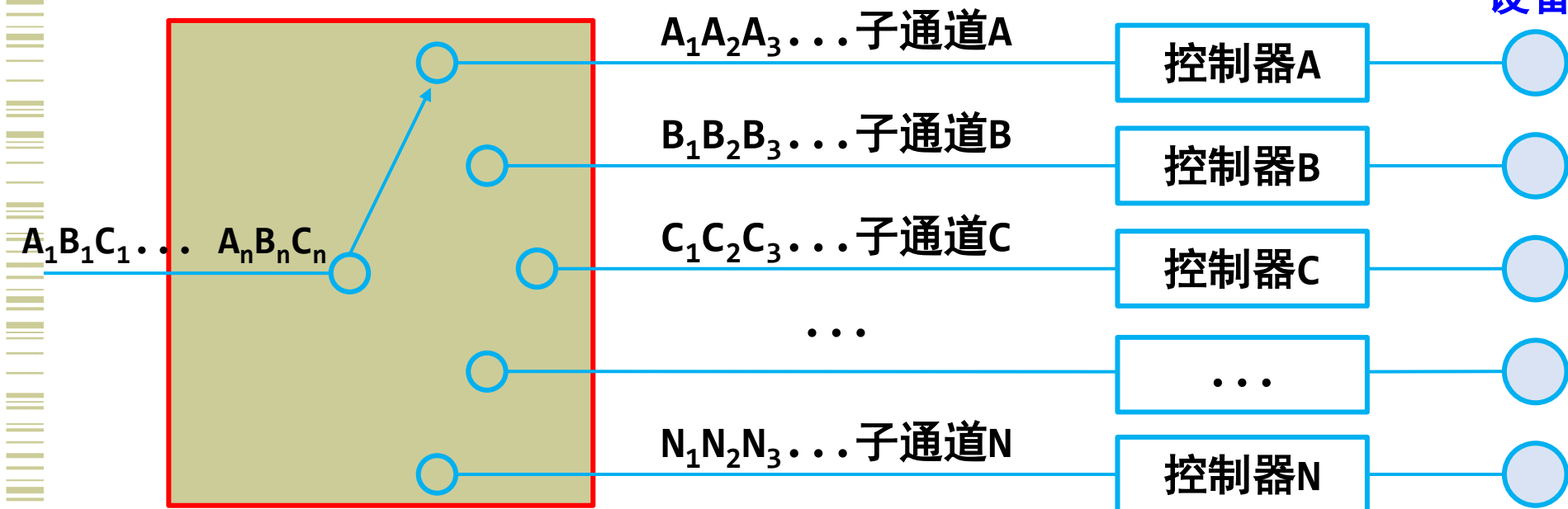


◆ 三种类型的通道与CPU、设备控制器和外设的连接关系

字节多路通道

- ◆ 为多台低速或中速的外设服务。
- ◆ 以字节交叉的方式**分时轮流**地为它们服务。
- ◆ 字节多路通道可以包含多个子通道，每个子通道连接一台设备控制器。
- ◆ 工作原理：**分时共享通道**，数据传输时才占用通道

设备



字节多路通道

◆ 工作方式：

◆ 字节交叉方式(byte-interleave mode)：

- 设备占用通道的时间片一定。很短的时间片内传输一个字节，或者说，不同的设备与通道在逻辑上建立了不同的传输连接，但在它所分得的时间片内建立真正物理连接。

◆ 成组方式(block mode)：

- 设备占用通道的时间片根据需要不定。允许一个设备一次占用通道比较长的时间传输一组数据，或者说，设备与通道的连接可以根据需要维持到一组数据全部传送完成。

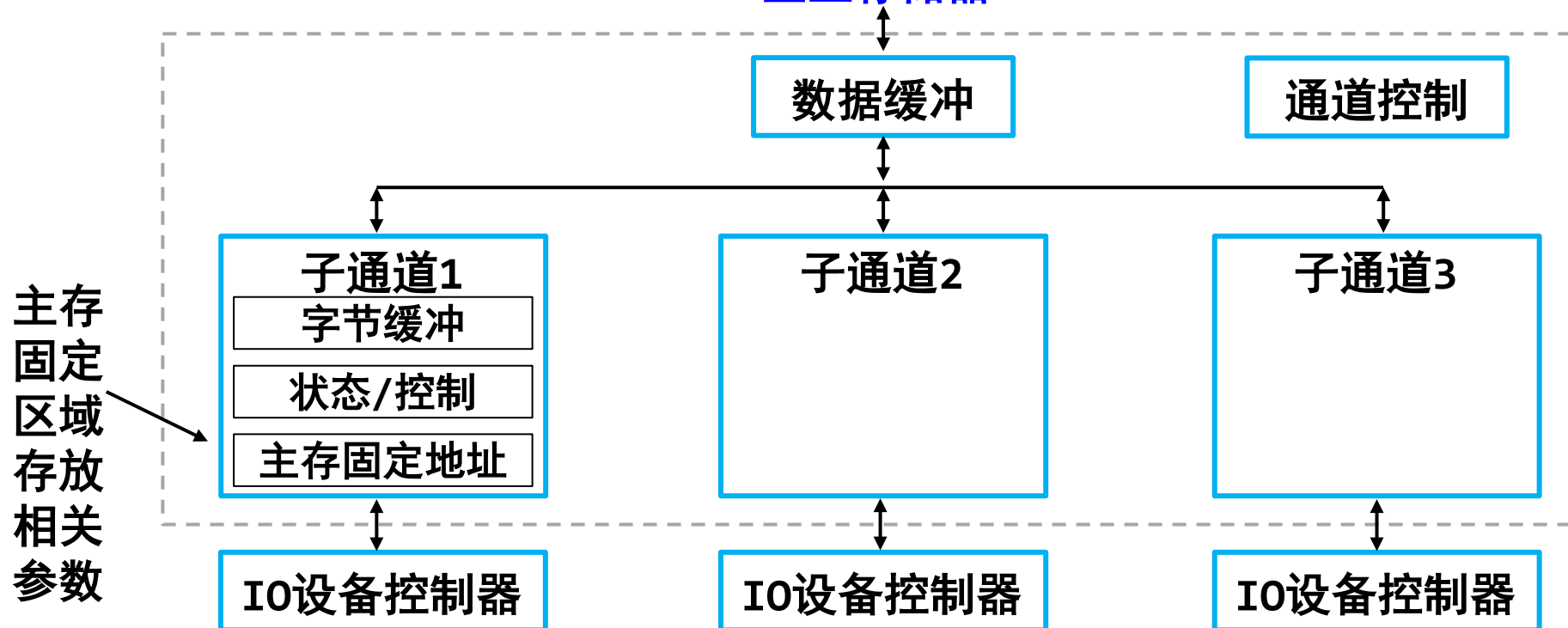
◆ 采用超时机制控制两种方式之间的自动转换

字节多路通道

◆ 字节多路通道的结构

- 包括多个子通道，每个子通道连接一个设备控制器。
- 控制部分以及与主存连接是公共的，每个子通道都有自己独立的一套寄存器。
- 主存数据缓冲区地址、交换字节个数等存放在主存的固定单元

至主存储器



选择通道

◆ 目的：

- 针对**高速外围设备**，在一段时间内单独为一台外围设备服务，在不同的时间内仍可以选择其他设备。

◆ 工作方式：

- **一旦选中某一设备，该外围设备就占用通道直到数据传输工作全部结束为止。**

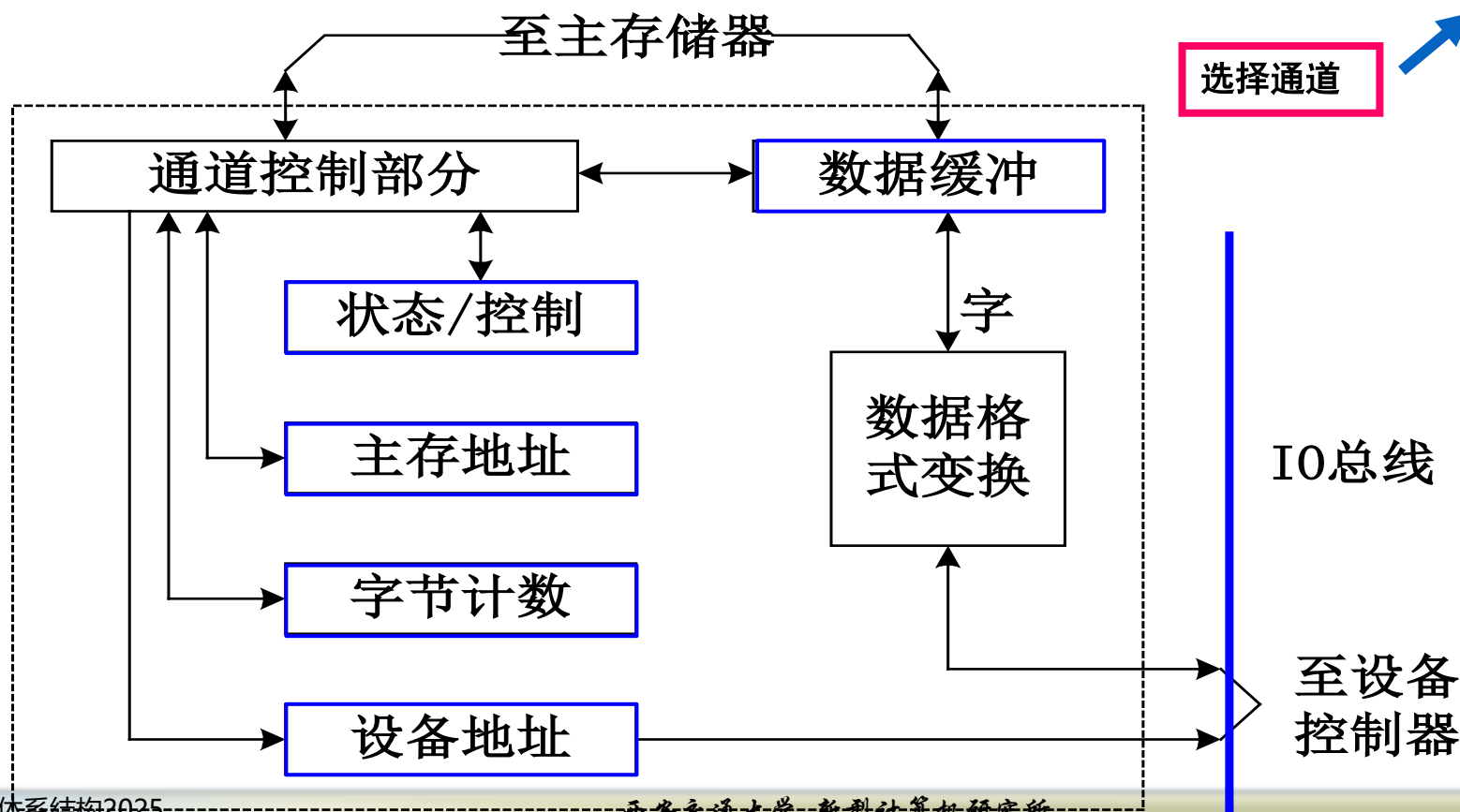
◆ 特点：

- 只有一个成组方式工作的子通道，只有一套完整的硬件，逐个为多台高速外围设备服务。
- 适合高速非同时要求数据传输的IO设备

选择通道

◆ 选择通道的结构：

- 5个寄存器：数据缓冲寄存器、设备地址寄存器、主存地址计数器、交换字节数计数器、设备状态/控制寄存器
- 格式变换部件和通道控制部分。



选择通道

- ◆ 问题：通道利用率不高，如读磁盘：
 - 第一步：定位。定位时间为几十毫秒
 - 第二步：寻找扇区。等待时间小于10毫秒
 - 第三步：读出数据。读数时间十几微秒

- ◆ 通道大部分时间在等待！ ！ ！

数组多路通道

◆ 目的：

- 前两种方式的结合，并发地为多台高速设备服务

◆ 工作方式：

- 通道在为一台高速外围设备传输数据时，有多台高速设备可以在定位或者找扇区。
- 每次选择一台高速设备，轮流为多台I/O设备服务。

◆ 工作步骤：对一台设备分多次操作

- 第一步：连接设备，发定位命令，断开连接。
- 第二步：连接设备，发寻找扇区命令，断开连接
- 第三步：连接设备，读出数据。

◆ 优点：数据传输率和通道的硬件利用率都很高。

◆ 缺点：控制硬件较复杂。

8.5.4 通道中的数据传送过程与流量分析

◆ 通道流量

- 一个通道在数据传送期间，单位时间内能够传送的数据量。所用单位一般为Bps。
- 又称为通道吞吐率、通道数据传输率等。

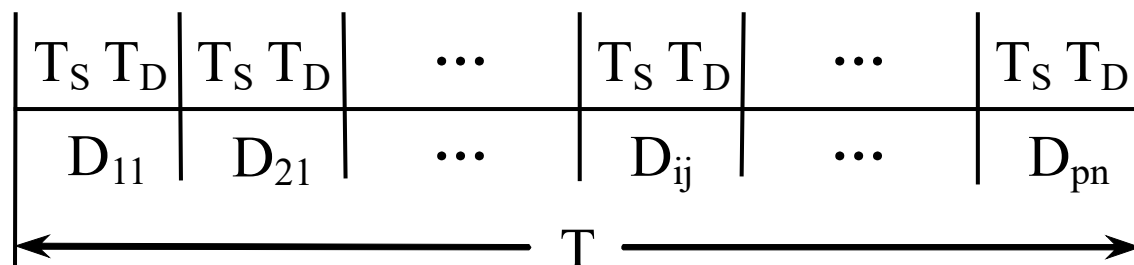
◆ 通道最大流量

- 一个通道在满负荷工作状态下的流量。

参数	含义	参数	含义
T_S	设备选择时间。从通道响应设备的数据传送请求，到实际为设备传送数据所需要的时间	n	每台设备传送的字节数，假设每台设备传送的字节数都相同
T_D	传送一个字节所用的时间	k	数组多路通道传输的一个数据块中包含的字节数。 $k < n$ ，通常 $k=512$
p	一个通道上连接的设备台数，且这些设备同时都在工作	T	通道完成全部数据传送工作所需要的时间

字节多路通道

◆ 数据传送过程



- ◆ D_{ij} : 连接在通道上的第*i*台设备传送的第*j*个数据
 - $i=1, 2, \dots, p$; $j=1, 2, \dots, n$
- ◆ 通道每连接一台个外设，只传送一个字节，然后又与另一台设备连接，并传送一个字节。

字节多路通道

- ◆ p 台设备每台传送 n 个数据总共所需的时间为

$$T_{\text{BYTE}} = (T_S + T_D) \times p \times n$$

- ◆ 最大流量

$$f_{\text{MAX-BYTE}} = \frac{pn}{(T_S + T_D)pn} = \frac{1}{T_S + T_D}$$

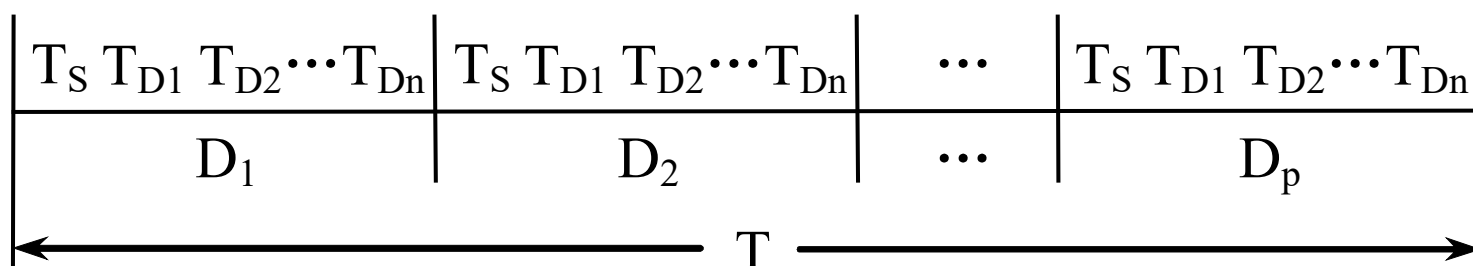
- ◆ 实际流量是连接在这个通道上的所有设备的数据传输率之和。

$$f_{\text{BYTE}} = \sum_{i=1}^p f_i$$

- f_i : 第 i 台设备的实际数据传输率

选择通道

- ◆ 在一段时间内只能单独为一台高速外设服务，当这台设备的数据传送工作全部完成后，通道才能为另一台设备服务。
- ◆ 工作过程



- ◆ 其中：
 - D_i 表示通道正在为第*i*台设备服务
 - $T_{D1} = T_{D2} = \cdots = T_{Dn} = T_D$

选择通道

- ◆ 1台设备传送 n 个数据总共所需的时间

$$T = T_S + T_D \times n = \left(\frac{T_S}{n} + T_D \right) \times n$$

- ◆ p 台设备每台传送 n 个数据总共所需的时间

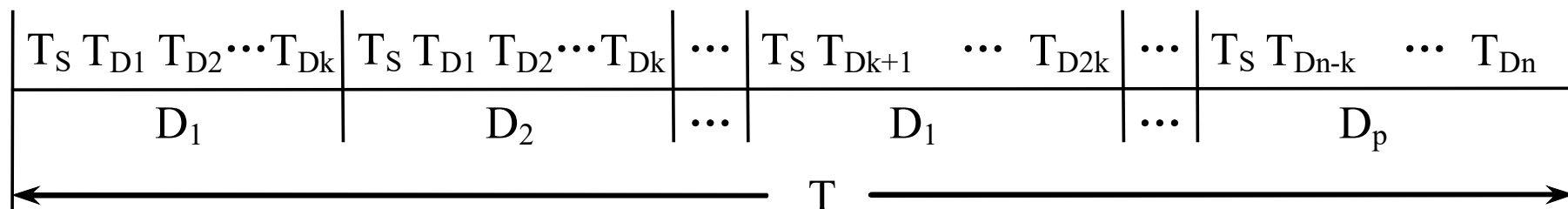
$$T_{\text{SELECT}} = T \times p = \left(\frac{T_S}{n} + T_D \right) \times n \times p = \left(\frac{T_S}{n} + T_D \right) pn$$

- ◆ 最大流量

$$f_{\text{MAX-SELECT}} = \frac{pn}{\left(\frac{T_S}{n} + T_D \right) pn} = \frac{1}{\frac{T_S}{n} + T_D}$$

数组多路通道

- ◆ 数组多路通道在一段时间内只能为一台高速设备传送数据，但同时可以有多台高速设备在寻址，包括定位和找扇区。
- ◆ 不考虑同时启动其他设备的优化情况下：



- ◆ 其中：
 - D_i 表示通道正在为第 i 台设备服务
 - $T_{D1} = T_{D2} = \cdots = T_{Dk} = T_D$
 - k : 一个数据块中的字节个数。在一般情况下, $k < n$ 。对于磁盘, 磁带等磁表面存储器, $k=512$ 。

数组多路通道

- ◆ 每台设备传送一个块（k个数据）所需的时间

$$T_K = T_S + T_D \times k = \left(\frac{T_S}{k} + T_D \right) \times k$$

- ◆ 每台设备传送一个数据所需的时间为

$$T' = \frac{T_K}{k} = \frac{\left(\frac{T_S}{k} + T_D \right) \times k}{k} = \frac{T_S}{k} + T_D$$

- ◆ p台设备每台传送n个数据总共所需的时间为：

$$T_{\text{BLOCK}} = T' \times p \times n = \left(\frac{T_S}{k} + T_D \right) \times p \times n$$

- ◆ 最大流量

$$f_{\text{MAX-BLOCK}} = \frac{pn}{\left(\frac{T_S}{k} + T_D \right) pn} = \frac{1}{\frac{T_S}{k} + T_D}$$

实际流量

- ◆ 选择通道和数组多路通道的实际流量就是连接在这个通道上的所有设备中数据流量最大的那一个

$$f_{\text{SELECT}} \leq \max_{i=1}^p f_i$$

$$f_{\text{BLOCK}} \leq \max_{i=1}^p f_i$$

- ◆ 各种通道的实际流量应该不大于通道的最大流量

$$f_{\text{BYTE}} \leq f_{\text{MAX-BYTE}}$$

$$f_{\text{SELECT}} \leq f_{\text{MAX-SELECT}}$$

$$f_{\text{BLOCK}} \leq f_{\text{MAX-BLOCK}}$$

- 两边的差值越小，通道的利用率就越高。
- 当两边相等时，通道处于满负荷工作状态。

字节多路通道-举例

- ◆ 例子：一个字节多路通道连接 D_1 、 D_2 、 D_3 、 D_4 、 D_5 共5台设备分别每10微秒、30微秒、30微秒、50微秒和75微秒向通道发出一次数据传送的服务请求，
- ◆ 求
 - 1) 计算这个字节多路通道的实际流量和工作周期。
 - 2) 画出通道分时为各个设备服务的时间关系图。
 - 3) 从时间关系图上发现什么问题？如何解决？

- ◆ 解：1) 该通道的实际流量为：

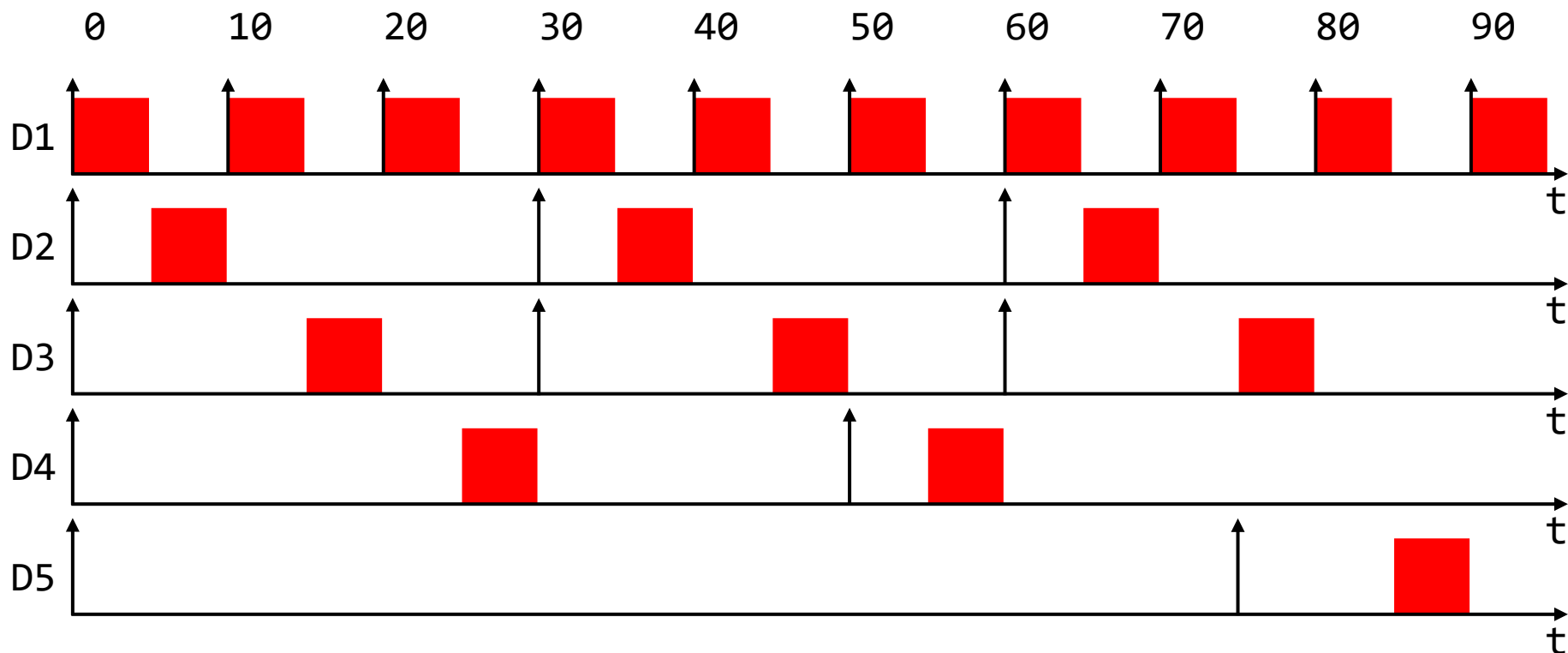
$$f_{\text{BYTE}} = \left(\frac{1}{10} + \frac{1}{30} + \frac{1}{30} + \frac{1}{50} + \frac{1}{75} \right) \text{MB/S} = 0.2 \text{MB/S}$$

该通道的工作周期（传送一个字节所需时间）为：

$$t_{\text{BYTE}} = \frac{1}{f_{\text{BYTE}}} = 5 \text{us/byte}$$

字节多路通道-举例

- ◆ 2) 5台设备向通道请求传送数据和通道为它们服务的时间关系如下图所示:



- ◆ 3) D5的第一次请求没有得到响应!

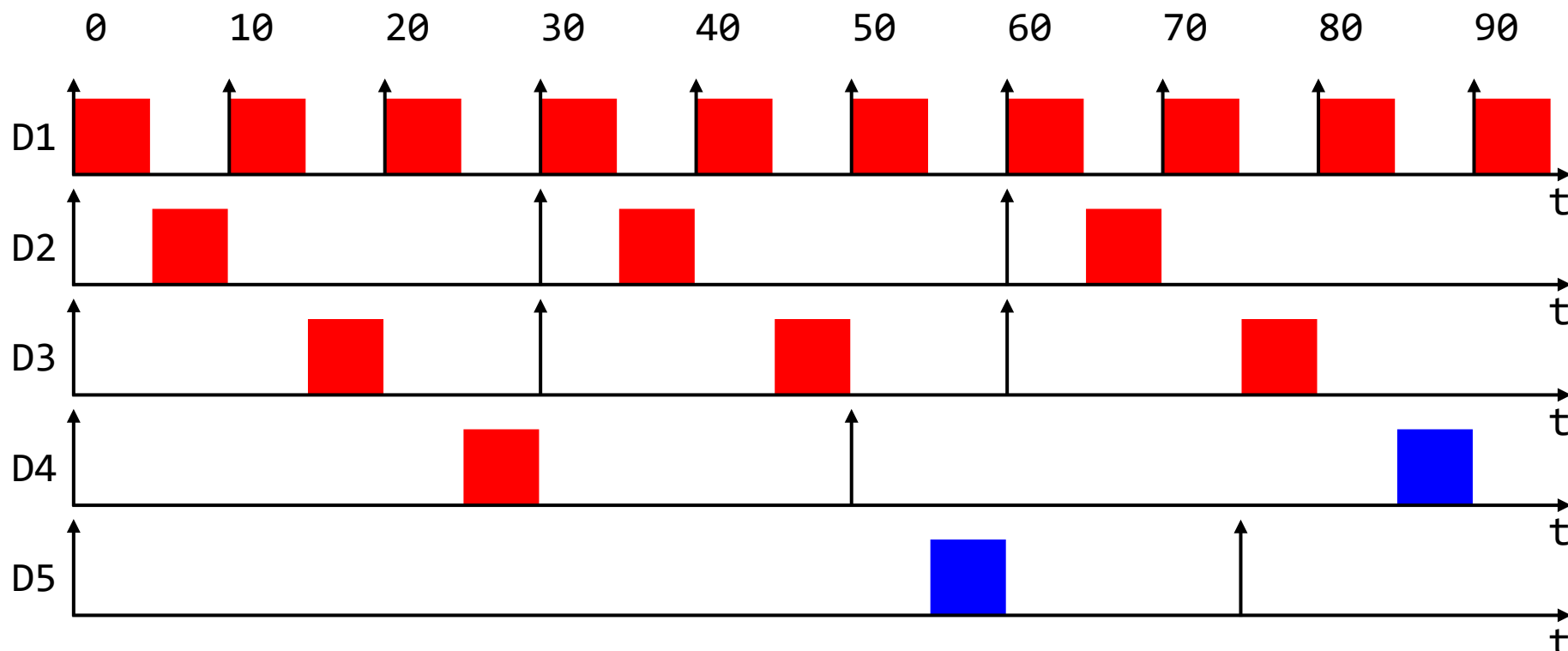
字节多路通道-举例

- ◆ D5的第一次请求没有得到响应！为什么？
- ◆ 问题分析
 - 当字节多路通道的最大流量与实际很接近时，虽然在宏观上保证通道流量平衡，不会丢失数据，但将因传输速度高的设备频繁发出请求，并优先得到通道的响应，而影响低速设备的请求服务。如D5设备。
- ◆ 解决思路：
 - 原则上，如果对所有设备的请求时间间隔取最小公倍数，则在这段时间内，通道的流量是平衡的，即：所有设备的请求都能得到一次响应服务。
 - 但是，这并不能保证在任意设备的任意两次时间传输请求之间的请求都能得到相应。例如：D5

字节多路通道-举例

◆ 解决策略

- 增加通道的最大流量
- 动态改变设备的优先级。
- 增加一定数量的数据缓冲区。

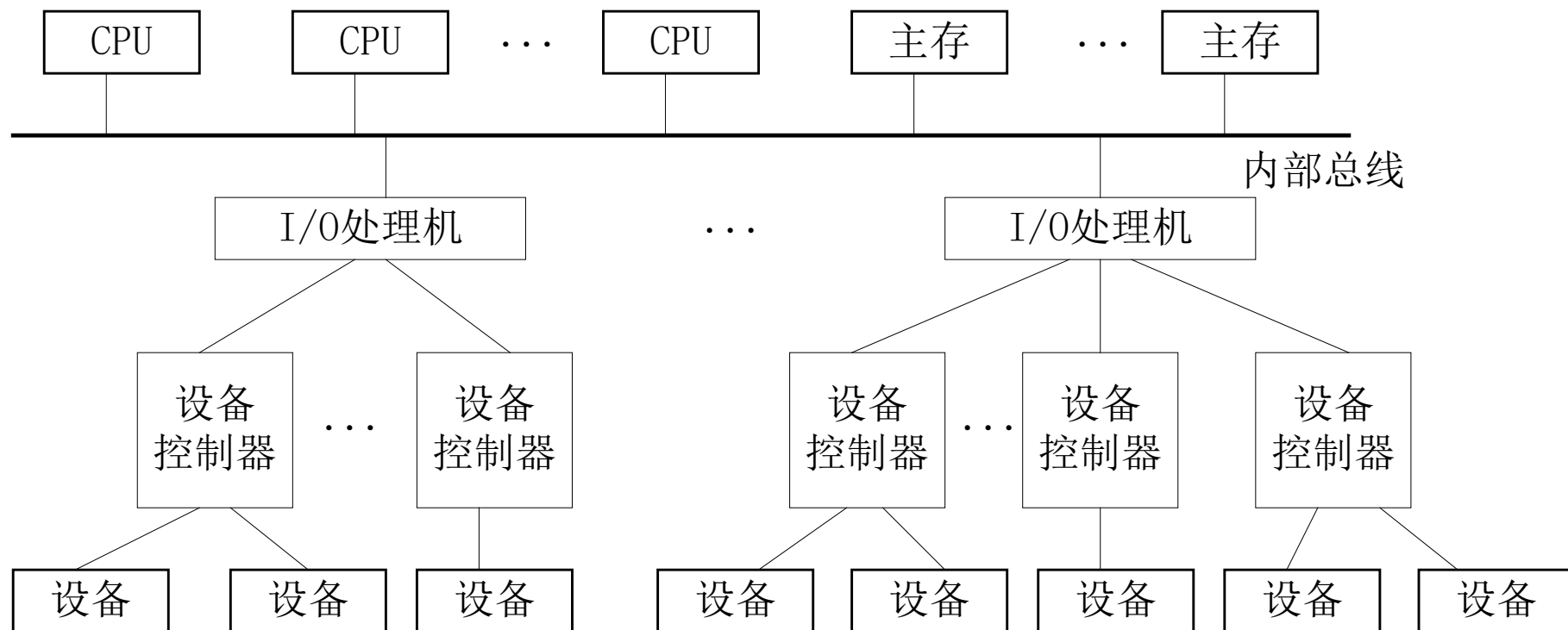


存在的问题

- ◆ 在巨型和大型计算机系统中，采用通道处理机仍然**存在高性能处理机利用率不高**的问题：
 - 每调用一次输入输出操作的前处理和后处理仍然要CPU来完成，需要两次中断方式中断CPU的现行程序。
 - 当外围设备或通道处理机出现异常情况时，通道处理机本身不能处理，要通过中断方式请求CPU来处理。
 - 对所有传送数据的格式转换、码制转换、数据块整体的正确性检验等工作仍然要CPU来完成。
 - 文件的管理、设备的管理等操作系统的工作，通道处理机本身无能为力，需要CPU来实现。
- ◆ 为了解决以上问题，通道处理机就需要象一般的处理机一样具有算术运算和程序控制等功能
 - **输入输出处理机**

输入输出处理机的作用

- ◆ 一台输入输出处理机可以管理多台设备控制器，一台设备控制器可以管理多台同类型的输入输出设备。如下图：



一种典型的采用输入输出处理机的计算机结构

输入输出处理机的作用

- ◆ IO处理机除了能够完成通道处理机的全部功能之外，还具有以下功能：
 - 码制转换
 - 数据校验和校正
 - 故障处理
 - 文件管理
 - 诊断和显示IO系统状态
 - 处理人机对话
 - 网络或远程终端可以连接到IO处理机上，由IO处理机完成远程用户服务工作

IO处理机的种类

- ◆ 与VLSI技术发展有关
- ◆ 根据**是否共享主存储器**，可以把输入输出处理机分为：
 - **共享主存储器的输入输出处理机**
 - 每个输入输出处理机具有小容量的局部存储器
 - 运行时，管理程序从主存储器加载到局部存储器
 - **不共享主存储器的输入输出处理机**
 - 每个输入输出处理机具有大容量的局部存储器
 - 管理程序放在局部存储器中
 - 减少了主存储器的压力

IO处理机的种类

- ◆ 根据**是否共享运算部件和指令控制部件**，可分为：
 - **合用同一个运算部件**和指令控制部件的输入输出处理机
 - 通过公用部件与主存储器连接
 - 成本低
 - 控制复杂
 - **独立运算部件**和指令控制部件的输入输出处理机
 - 具有大容量局部存储器
 - 独立性很强
 - 是目前的主流

IO处理机的种类

- ◆ 根据各种计算机系统的具体情况和不同要求，输入输出处理机的结构的组织方式，可分为：
 - 有多个输入输出处理机，而且从功能上进行分工
 - 每个输入输出处理机完成专门的管理
 - 以输入输出处理机作为主处理机，除了担负全部输入输出工作外还运行操作系统。
 - 一般在并行计算机中采用
 - 用一台与中央处理机相同型号的处理机作为输入输出处理机。
 - 输入输出处理机也可参与计算
 - 采用廉价的微处理器来专门承担输入输出任务。

8.6 I/O与操作系统

- ◆ 设计I/O系统需要注意操作系统的因素
 - I/O操作主要是在外设和存储器之间进行，所以操作系统必须**保证这些I/O操作的安全性**。
- ◆ 8.6.1 DMA和虚拟存储器
- ◆ DMA是使用虚拟地址还是物理地址？
- ◆ **使用物理地址进行DMA传输，存在以下两个问题：**
 - 对于超过一页的数据缓冲区，由于缓冲区使用的页面在物理存储器中不一定是连续的，所以传输可能会发生问题。
 - 如果DMA正在存储器和缓冲区之间传输数据时，操作系统从存储器中移出（或重定位）一些页面，那么，DMA将会在存储器中错误的物理页面上进行数据传输。

8.6.1 DMA和虚拟存储器

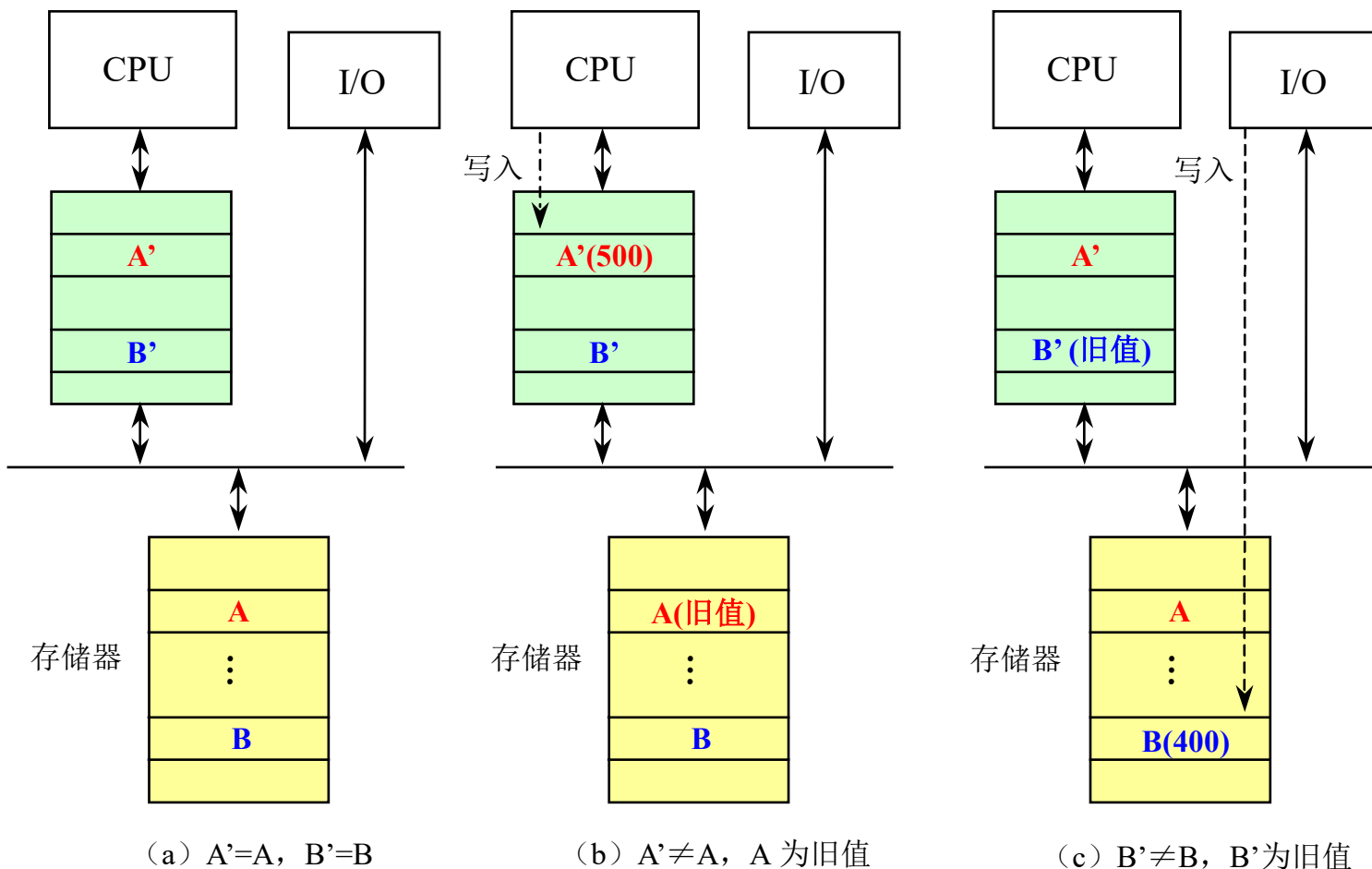
- ◆ 解决这些问题的方法
- ◆ 使操作系统在I/O的传输过程中确保DMA设备所访问的页面都位于物理存储器中，这些页面被称为是**钉**在了主存中。
- ◆ **“虚拟DMA” 技术**
 - 允许DMA设备直接使用虚拟地址，并在DMA期间由硬件将虚拟地址转换为物理地址。
 - 在采用虚拟DMA的情况下，如果进程在内存中被移动，操作系统应该能够及时地修改相应的DMA地址表。

8.6.2 I/O和Cache数据一致性

- ◆ Cache会使一个数据出现两个副本：
 - 一个在Cache中，另一个在主存中。
- ◆ I/O设备可以修改存储器中的内容
- ◆ 把I/O连接到存储器上
 - CPU修改了Cache的内容后，由于存储器的内容跟不上Cache内容的变化，I/O系统进行输出操作时所看到的数据是旧值。（写直达Cache没有这样的问题）
 - I/O系统进行输入操作后，存储器的内容发生了变化，但CPU在Cache中所看到的内容依然是旧值。
- ◆ 把I/O直接连接到Cache上

8.6.2 I/O和Cache数据一致性

- ◆ 举例：假设Cache采用写回法，并且A'是A的副本，B'是B的副本。



8.6.2 I/O和Cache数据一致性

◆ 把I/O直接连接到Cache上

- 不会产生由I/O导致的数据不一致的问题。
- 所有I/O设备和CPU都能在Cache中看到最新的数据。
- I/O会跟CPU竞争访问Cache，在进行I/O时，会造成CPU的停顿。
- I/O还可能会破坏Cache中CPU访问的内容，因为I/O操作可能导致一些新数据被加入Cache，而这些新数据可能在近期内并不会被CPU访问。

解决内容一致性问题方法

◆ 软件的方法

- 设法保证I/O缓冲器中的所有各块都不在Cache中。
- 具体做法有两种
 - 把I/O缓冲器的页面设置为不可进入Cache的，在进行输入操作时，操作系统总是把输入的数据放到该页面上
 - 在进行输入操作之前，操作系统先把Cache中与I/O缓冲器相关的数据“赶出”Cache，即把相应的数据块设置为“无效”状态。

◆ 硬件的方法

- 在进行输入操作时，检查相应的I/O地址（I/O缓冲器中的单元）是否在Cache中（即是否有数据副本）。
- 如果发现I/O地址在Cache中有匹配的项，就把相应的Cache块设置为“无效”。